

final-melhorado-v2

June 10, 2026

```
[1]: '''  
      Bibliotecas usadas  
      '''  
  
      import os  
      import re  
      import subprocess  
      import warnings  
      import numpy as np  
      import pandas as pd  
      from collections import Counter  
      from Bio import Entrez  
      from Bio import SeqIO  
      from Bio.Seq import Seq  
      from sklearn.model_selection import GroupShuffleSplit, StratifiedGroupKFold  
      from sklearn.metrics import (  
          classification_report,  
          roc_auc_score,  
          average_precision_score,  
          make_scorer,  
          matthews_corrcoef,  
          f1_score,  
          precision_score,  
          recall_score,  
          balanced_accuracy_score,  
          confusion_matrix,  
          ConfusionMatrixDisplay,  
          roc_curve,  
          auc  
      )  
  
      from sklearn.ensemble import RandomForestClassifier  
      from sklearn.model_selection import cross_validate  
      from sklearn.svm import SVC  
      from sklearn.cluster import KMeans  
      from sklearn.preprocessing import StandardScaler  
      from xgboost import XGBClassifier
```

```

import matplotlib.pyplot as plt
import subprocess
import shap
import seaborn as sns
from sklearn.inspection import permutation_importance

warnings.filterwarnings("ignore")

print("Bibliotecas importadas")

```

Bibliotecas importadas

```

[2]: '''
    obtenção dos datasets
    '''

EMAIL = "marcela.leite@gmail.com.br"
OUTPUT_DIR = "pipelineW"
os.makedirs(OUTPUT_DIR, exist_ok=True)
Entrez.email = EMAIL

# =====
# QUERIES
# =====

# POSITIVE_QUERY = r'''
# ("Solanum lycopersicum"[Organism] OR "Nicotiana benthamiana"[Organism] OR
# ↪ "Capsicum annuum"[Organism] OR "Arabidopsis thaliana"[Organism])
# AND( TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA
# ↪ OR PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
# OR "virus resistance" OR "antiviral" OR "RNA silencing" OR "RDR" OR
# ↪ "Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" OR "R gene" OR "Sw-5" OR
# ↪ "Tm-2" OR "RCY1"
# OR "Rx" OR "R protein" OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR"
# ↪ OR "CC-NBS-LRR" OR "RLK" OR "RLP" OR "LRR receptor kinase"
# OR "leucine rich repeat receptor kinase" OR "plant disease resistance
# ↪ protein" OR "disease resistance protein" OR "RPM1" OR "RPS2" OR "RPS4" OR
# ↪ "RPS5"
# OR "RPP" OR "SNC1" OR "ADR1" OR "NDR1" OR "EDS1" OR "PAD4" )
# NOT ( partial OR fragment OR hypothetical OR predicted OR DAP-seq OR
# ↪ "transcription factor" OR DNA-binding )
# '''

POSITIVE_QUERY = r'''

```

```

    ("Solanaceae"[Organism] NOT ("Solanum betaceum"[Organism] OR "Solanum_
    ↪melongena"[Organism] OR "Solanum tuberosum"[Organism] OR "Physalis_
    ↪peruviana"[Organism]))
    AND (TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA_
    ↪OR PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
    OR "virus resistance" OR "antiviral" OR "RNA silencing" OR "RDR" OR_
    ↪"Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" OR "R gene" OR "Sw-5" OR_
    ↪"Tm-2" OR "RCY1"
    OR "Rx" OR "R protein" OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR"_
    ↪OR "CC-NBS-LRR" OR "RLK" OR "RLP" OR "LRR receptor kinase"
    OR "leucine rich repeat receptor kinase" OR "plant disease resistance_
    ↪protein" OR "disease resistance protein" OR "RPM1" OR "RPS2" OR "RPS4" OR_
    ↪"RPS5"
    OR "RPP" OR "SNC1" OR "ADR1" OR "NDR1" OR "EDS1" OR "PAD4" OR "NB-ARC" OR_
    ↪"NBS-ARC" OR "TIR domain"
    OR "NB-ARC domain" OR "NBS-ARC" OR "leucine-rich repeat domain" OR "TIR_
    ↪domain-containing" OR "LRR-containing" OR "coiled-coil leucine-rich repeat")
    NOT (partial OR fragment OR hypothetical OR predicted OR DAP-seq OR_
    ↪"transcription factor" OR DNA-binding OR microtom )
    ...
# #OR microtom
# NEGATIVE_QUERY = r'''
# ( "Solanum lycopersicum"[Organism] OR "Nicotiana benthamiana"[Organism] OR_
    ↪"Capsicum annuum"[Organism] OR "Arabidopsis thaliana"[Organism])
# AND ( "actin" OR "tubulin" OR "ribosomal protein" OR "ATP synthase" OR_
    ↪"photosystem" OR "elongation factor" OR "glycolysis" OR "metabolic enzyme" )
# NOT ( "virus resistance" OR "antiviral" OR "RNA silencing" OR "RDR" OR_
    ↪"Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" OR "R gene" OR "Sw-5" OR_
    ↪"Tm-2"
# OR "RCY1" OR "Rx" OR "R protein" OR "resistance" OR "NLR" OR "LRR" OR_
    ↪"immune" OR "virus" )
# '''
NEGATIVE_QUERY = r'''
    ("Solanaceae"[Organism] NOT ("Solanum melongena"[Organism] OR "Solanum_
    ↪tuberosum"[Organism] OR "Physalis peruviana"[Organism]))
    AND 100:3000[Sequence Length]
    NOT (TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA_
    ↪OR PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
    OR "virus resistance" OR "antiviral" OR "RNA silencing" OR "RDR" OR_
    ↪"Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" OR "R gene" OR "Sw-5" OR_
    ↪"Tm-2" OR "RCY1"
    OR "Rx" OR "R protein" OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR"_
    ↪OR "CC-NBS-LRR" OR "RLK" OR "RLP" OR "LRR receptor kinase"
    OR "leucine rich repeat receptor kinase" OR "plant disease resistance_
    ↪protein" OR "disease resistance protein" OR "RPM1" OR "RPS2" OR "RPS4" OR_
    ↪"RPS5"

```

```

    OR "RPP" OR "SNC1" OR "ADR1" OR "NDR1" OR "EDS1" OR "PAD4")
    NOT (partial OR fragment OR hypothetical OR predicted OR uncharacterized OR_
↳DAP-seq OR "transcription factor" OR DNA-binding OR microtom )
'''

# Dados dos transcriptomas
ARABIDOPSIS_QUERY = '''
    "Arabidopsis thaliana"[Organism] AND
        (transcriptome OR RNA-Seq OR TSA OR leaf OR fruit OR root OR stress OR_
↳virus OR infected)
        NOT (genome OR chromosome OR scaffold OR contig OR partial OR fragment OR_
↳microtom)
        AND 1500:30000[Sequence Length]
'''

SOLANUM_MELONGENA = '''
    Solanum melongena [Organism]
AND (transcriptome OR RNA-Seq OR TSA OR leaf OR fruit OR root OR stress OR_
↳virus OR infected)
        NOT (genome OR chromosome OR scaffold OR contig OR partial OR fragment_
↳OR microtom)
        AND 1500:30000[Sequence Length]
'''

TUBEROSUM_QUERY = '''
    Solanum tuberosum [Organism]
AND (transcriptome OR RNA-Seq OR TSA OR leaf OR fruit OR root OR stress OR_
↳virus OR infected)
        NOT (genome OR chromosome OR scaffold OR contig OR partial OR fragment_
↳OR microtom)
        AND 1500:30000[Sequence Length]
'''

PHYSALIS_QUERY = '''
    "Physalis peruviana"[Organism] AND
        (transcriptome OR RNA-Seq OR TSA OR leaf OR fruit OR root OR stress OR_
↳virus OR infected)
        NOT (genome OR chromosome OR scaffold OR contig OR partial OR fragment OR_
↳microtom)
        AND 1500:30000[Sequence Length]
'''

# =====
# DOWNLOAD NCBI
# =====

```

```

def baixar_proteinas_ncbi(query, output_fasta, max_records=15000):
    print("\n=====")
    print("DOWNLOAD NCBI: ",output_fasta)
    print("=====")
    handle = Entrez.esearch(
        db="protein",
        term=query,
        retmax=max_records
    )
    record = Entrez.read(handle)
    ids = record["IdList"]
    print(f"Proteínas encontradas: {len(ids)}")
    if len(ids) == 0:
        return
    handle = Entrez.efetch(
        db="protein",
        id=ids,
        rettype="fasta",
        retmode="text"
    )
    with open(output_fasta, "w") as f:
        f.write(handle.read())
    print(f"Arquivo salvo: {output_fasta}")

def baixar_transcriptoma(output_fasta, query):
    print("\n=====")
    print("DOWNLOAD TRANSCRIPTOMA: ",output_fasta)
    print("=====")
    handle = Entrez.esearch(
        db="nucleotide",
        term=query,
        retmax=10000
    )
    record = Entrez.read(handle)
    ids = record["IdList"]
    print(f"Transcritos encontrados: {len(ids)}")
    handle = Entrez.efetch(
        db="nucleotide",
        id=ids,
        rettype="fasta",
        retmode="text"
    )
    with open(output_fasta, "w") as f:
        f.write(handle.read())
    print(f"Arquivo salvo: {output_fasta}")

```

```

#Dados de Entrada - baixar arquivos FASTA das sequências de proteínas
POSITIVE_FASTA = f"{OUTPUT_DIR}/positive.fasta"
NEGATIVE_FASTA = f"{OUTPUT_DIR}/negative.fasta"

baixar_proteinas_ncbi(POSITIVE_QUERY, POSITIVE_FASTA)
baixar_proteinas_ncbi(NEGATIVE_QUERY, NEGATIVE_FASTA)

print("Arquivos salvos na pasta: ", OUTPUT_DIR)
print("===== FIM COLETA DE DADOS TREINO/VALIDAÇÃO =====")

```

```

=====
DOWNLOAD NCBI: pipelineW/positive.fasta
=====
Proteínas encontradas: 13627
Arquivo salvo: pipelineW/positive.fasta

=====
DOWNLOAD NCBI: pipelineW/negative.fasta
=====
Proteínas encontradas: 15000
Arquivo salvo: pipelineW/negative.fasta
Arquivos salvos na pasta: pipelineW
===== FIM COLETA DE DADOS TREINO/VALIDAÇÃO =====

```

```

[4]: #Dados para aplicação - baixar transcriptomas - RNA-Seq
transcriptoma_arabidopsis_fasta = (f"{OUTPUT_DIR}/arabidopsis_transcriptoma.
↳fasta")
transcriptoma_melongena_fasta = (f"{OUTPUT_DIR}/melongena_transcriptoma.fasta")
transcriptoma_tuberosum_fasta = (f"{OUTPUT_DIR}/tuberosum_transcriptoma.fasta")
transcriptoma_physalis_fasta = (f"{OUTPUT_DIR}/physalis_transcriptoma.fasta")

# baixar_transcriptoma(transcriptoma_arabidopsis_fasta, ARABIDOPSIS_QUERY)
# baixar_transcriptoma(transcriptoma_melongena_fasta, SOLANUM_MELONGENA)
# baixar_transcriptoma(transcriptoma_tuberosum_fasta, TUBEROSUM_QUERY)
baixar_transcriptoma(transcriptoma_physalis_fasta, PHYSALIS_QUERY)
print("Arquivos salvos na pasta: ", OUTPUT_DIR)
print("===== FIM COLETA DE DADOS APLICAÇÃO =====")

```

```

=====
DOWNLOAD TRANSCRIPTOMA: pipelineW/physalis_transcriptoma.fasta
=====
Transcritos encontrados: 10000
Arquivo salvo: pipelineW/physalis_transcriptoma.fasta
Arquivos salvos na pasta: pipelineW

```

===== FIM COLETA DE DADOS APLICAÇÃO =====

```
[5]: '''
    Tratamento dos dados
        - Criar Dataframes com os conjuntos de dados de treino e validação
        - Executar CD-Hit para eliminar sequencias muito semelhantes
        - Extração de atributos (features)

    '''

# =====
# AAC
# =====
# composição de aminoácidos - transformar a proteína em dados numéricos
# conta a frequência de cada aminoácido em uma sequência de proteína
# como são 20 aminoácidos possíveis, irá gerar 20 features com seus percentuais
# com essa frequência consegue ter um parâmetro para distinguir as proteínas
# por exemplo proteínas de resistência tem geralmente certa frequência de
    ↳ aminoácidos
# mas ignora a ordem - por isso a sequência de dipeptídeos irá incrementar o
    ↳ modelo considerando a ordem
# sequência de 20 aminoácidos
AMINO_ACIDOS = list("ACDEFGHIKLMNPQRSTVWY")

motifs = {
    # "P_LOOP": r"[A-Z]G[A-Z]{4}GK[ST]",
    # "GLPL": r"GLPL",
    # "MHD": r"MHD"

    # # novos motifs
    # "KINASE2": r"[LIVM]{3,4}DD[VLIM]",
    # "RNBS_A": r"FDLxKxWVSVD",
    # "RNBS_B": r"[A-Z]{2}G[PH][A-Z]{2}",
    # "RNBS_C": r"[A-Z]{2}P[A-Z]{2}W",
    # "RNBS_D": r"CFLYCALFPED",
    # "VVVLDDVW": r"VVVLDDVW"
    # "P_LOOP": r"[A-Z]G[A-Z]{4}GK[ST]",
    # "P_LOOP": r"G[PAST][A-Z]{4}GK[ST]",
    "P_LOOP": r"G[A-Z]{4}GK[ST]",
    # "KINASE2": r"[LIVM]{3,5}DD",
    "KINASE2": r"[LIVM]{4}DD[VIL]",
    "RNBS_A": r"FDL",
    "RNBS_B": r"G[RK]G",
    "RNBS_C": r"[A-Z]{2}P[A-Z]{2}W",
    "GLPL": r"GLPL",
    "RNBS_D": r"CFL",
    "MHD": r"MHD"
```

```

}

def calcular_aac(seq):
    seq = seq.upper()
    total = len(seq)
    if total == 0:
        return [0] * len(AMINO_ACIDOS)
    counts = Counter(seq)
    return [
        counts.get(aa, 0) / total
        for aa in AMINO_ACIDOS
    ]

# =====
# DIPEPTÍDEOS
# =====
'''
encontrar pares de aminoácidos consecutivos em uma sequência protéica □
↳-sequência de dois aminoácidos
considera a ordem dos aminoácidos
há certos padrões comuns para
para 20 aminoácidos, há possibilidade de 20*20 = 400 dipeptídeos - features
'''

DIPEPTIDES = [
    a + b
    for a in AMINO_ACIDOS
    for b in AMINO_ACIDOS
]

def calcular_dipeptideos(seq):
    seq = seq.upper()
    total = len(seq) - 1
    if total <= 0:
        return [0] * len(DIPEPTIDES)
    counts = Counter([
        seq[i:i+2]
        for i in range(total)
    ])
    return [
        counts.get(dp, 0) / total
        for dp in DIPEPTIDES
    ]

def localizar_motivos(seq):
    posicoes = {}
    for nome, pattern in motifs.items():

```



```

        match = re.search(pattern, seq)

        if match:
            posicoes[nome] = match.start()
        else:
            posicoes[nome] = None

    return posicoes

def calcular_distancias(posicoes):

    def distancia(a, b):
        if posicoes[a] is None or posicoes[b] is None:
            return -1
        return posicoes[b] - posicoes[a]

    return {
        "dist_ploop_kinase2": distancia("P_LOOP", "KINASE2"),
        "dist_kinase2_glpl": distancia("KINASE2", "GLPL"),
        "dist_glpl_mhd": distancia("GLPL", "MHD")
    }

def contar_motivos(posicoes):
    return sum(
        1 for v in posicoes.values()
        if v is not None
    )

def extrair_motifs(seq):

    return [
        int(bool(re.search(pattern, seq)))
        for pattern in motifs.values()
    ]

# =====
# FEATURES - atributos
# =====
#calculando features
def extrair_features_proteina(seq):
    seq = re.sub(r"[^ACDEFGHIKLMNPQRSTVWY]", "", seq)
    if len(seq) < 50:
        return None
    features = []

```

```

features.append(len(seq)) # 1 - comprimento da cadeia
features.extend(calcular_aac(seq)) # 20 composição de aminoácidos -
↳ frequência de cada um na sequência
features.extend(calcular_dipeptideos(seq)) #400 possibilidades - considera
↳ a ordem dos aminoácidos para buscar os mais frequentes em proteínas de
↳ resistência

features.extend(extrair_motifs(seq))
# posições dos motivos
posicoes = localizar_motivos(seq)
# número de motivos encontrados
num_motifs_detectados = contar_motivos(posicoes)
# distâncias entre motivos
distancias = calcular_distancias(posicoes)

features.append(num_motifs_detectados)
protein_length = len(seq)

features.append(distancias["dist_ploop_kinase2"])
features.append(distancias["dist_kinase2_glpl"])
features.append(distancias["dist_glpl_mhd"])

features.append(
    distancias["dist_ploop_kinase2"]/protein_length
    if distancias["dist_ploop_kinase2"] != -1 else -1
)
features.append(
    distancias["dist_kinase2_glpl"]/protein_length
    if distancias["dist_kinase2_glpl"] != -1 else -1
)
features.append(
    distancias["dist_glpl_mhd"]/protein_length
    if distancias["dist_glpl_mhd"] != -1 else -1
)

features.append(int(posicoes["P_LOOP"] is not None))
features.append(int(posicoes["KINASE2"] is not None))
features.append(int(posicoes["GLPL"] is not None))
features.append(int(posicoes["MHD"] is not None))

return features

# =====
# DATASET
# =====
motif_cols = [
    "P_LOOP",

```

```

        "KINASE2",
        "RNBS_A",
        "RNBS_B",
        "RNBS_C",
        "GLPL",
        "RNBS_D",
        "MHD"
    ]

def criar_dataset(positive_fasta, negative_fasta):
    data = []
    columns = (
        ["protein_length"]
        + [f"AAC_{aa}" for aa in AMINO_ACIDOS]
        + [f"DIPEP_{dp}" for dp in DIPEPTIDES]
        + motif_cols
        + ["num_motifs_detectados"]
        + ["dist_ploop_kinase2"]
        + ["dist_kinase2_glpl"]
        + ["dist_glpl_mhd"]
        + ["dist_ploop_kinase2_norm"]
        + ["dist_kinase2_glpl_norm"]
        + ["dist_glpl_mhd_norm"]
        + ["has_ploop"]
        + ["has_kinase2"]
        + ["has_glpl"]
        + ["has_mhd"]
    )

    # POSITIVOS
    for record in SeqIO.parse(positive_fasta, "fasta"):
        seq = str(record.seq)
        feats = extrair_features_proteina(seq)
        if feats is None:
            continue
        row = feats + [1, record.id]
        data.append(row)

    # NEGATIVOS
    for record in SeqIO.parse(negative_fasta, "fasta"):
        seq = str(record.seq)
        feats = extrair_features_proteina(seq)
        if feats is None:
            continue
        row = feats + [0, record.id]
        data.append(row)

```

```

df = pd.DataFrame(
    data,
    columns=columns + ["target", "id"]
)
print("\nTotal de Features/Atributos:", len(columns)) # quantidade
print("\nFeatures/atributos:")
print(columns)
return df, columns

# CD-Hit
# chama o programa CD-HIT do sistema operacional
def executar_cdhit(
    input_fasta,
    output_fasta,
    identity=0.7 #0.5
):
    cmd = [
        "cd-hit",
        "-i", input_fasta,
        "-o", output_fasta,
        "-c", str(identity),
        "-n", "5",
        "-M", "30000",
        "-T", "4"
    ]

    print("\nExecutando CD-HIT...")
    subprocess.run(cmd, check=True)
    print("CD-HIT concluído.")

def limpar(seq_id):
    seq_id = seq_id.strip()
    seq_id = seq_id.split()[0]
    return seq_id

def ler_clusters_cdhit(cluster_file):
    clusters = {}
    cluster_id = None
    with open(cluster_file) as f:
        for line in f:
            line = line.strip()
            if line.startswith(">Cluster"):
                cluster_id = line.replace(">Cluster ", "")
                clusters[cluster_id] = []
            else:
                seq_id = line.split(">")[1].split("...")[0]
                seq_id = limpar(seq_id)

```

```

        clusters[cluster_id].append(seq_id)
    return clusters

# executa o CD-Hit nos arquivos baixados
executar_cdhit(
    POSITIVE_FASTA,
    f"{OUTPUT_DIR}/positive_nr.fasta",
    identity=0.7
)

executar_cdhit(
    NEGATIVE_FASTA,
    f"{OUTPUT_DIR}/negative_nr.fasta",
    identity=0.7
)

# =====
# Montar os DATASETS
# =====

POSITIVE_FASTA = f"{OUTPUT_DIR}/positive_nr.fasta"
NEGATIVE_FASTA = f"{OUTPUT_DIR}/negative_nr.fasta"

df, columns = criar_dataset(
    POSITIVE_FASTA,
    NEGATIVE_FASTA
)

df["id"] = df["id"].apply(limpar)
# =====
# CLUSTERS
# =====

clusters_pos = ler_clusters_cdhit(
    f"{OUTPUT_DIR}/positive_nr.fasta.clstr"
)

clusters_neg = ler_clusters_cdhit(
    f"{OUTPUT_DIR}/negative_nr.fasta.clstr"
)

cluster_map = {}

for c, seqs in clusters_pos.items():
    for s in seqs:
        cluster_map[s] = f"POS_{c}"

```

```

for c, seqs in clusters_neg.items():
    for s in seqs:
        cluster_map[s] = f"NEG_{c}"

df["cluster"] = df["id"].map(cluster_map)

print("\n Clusters ausentes: ")
print(df["cluster"].isna().sum())
df = df.dropna(subset=["cluster"])

print("\nDataset original pós CD-HIT:", df.shape)

# =====
# NOVO BLOCO: BALANCEAMENTO IMPERATIVO (DOWNSAMPLING)
# =====
print("\n--- Executando Balanceamento de Classes ---")
df_positivos = df[df["target"] == 1]
df_negativos = df[df["target"] == 0]

print(f"Contagem inicial -> Positivos (R-genes): {len(df_positivos)} |  

↳ Negativos (Background): {len(df_negativos)}")

# Se houver mais negativos do que positivos (cenário padrão), reduzimos os  

↳ negativos
if len(df_negativos) > len(df_positivos):
    # O sample() escolhe aleatoriamente N linhas do grupo majoritário
    df_negativos_balanceados = df_negativos.sample(n=len(df_positivos),  

↳ random_state=42)
    df = pd.concat([df_positivos, df_negativos_balanceados]).  

↳ reset_index(drop=True)
    print(f"Subamostragem aplicada com sucesso nos Negativos.")
else:
    print("O dataset já está balanceado ou possui mais positivos que negativos  

↳ (incomum). Nenhuma ação foi tomada.")

print(f"Contagem final -> Positivos: {len(df[df['target'] == 1])} | Negativos:  

↳ {len(df[df['target'] == 0])}")
print("Dataset Final Pronto:", df.shape)

print("Fim tratamento dos dados...")

```

Executando CD-HIT...

```

=====
Program: CD-HIT, V4.8.1 (+OpenMP), Aug 20 2021, 08:39:56
Command: cd-hit -i pipelineW/positive.fasta -o
pipelineW/positive_nr.fasta -c 0.7 -n 5 -M 30000 -T 4

```

Started: Wed Jun 10 09:31:30 2026

=====

Output

total seq: 9999
longest and shortest : 3540 and 29
Total letters: 7335929
Sequences have been sorted

Approximated minimal memory consumption:

Sequence : 8M
Buffer : 4 X 11M = 45M
Table : 2 X 65M = 131M
Miscellaneous : 0M
Total : 185M

Table limit with the given memory limit:

Max number of representatives: 4000000
Max number of word counting entries: 3726847813

comparing sequences from 0 to 1666
----- new table with 244 representatives
comparing sequences from 1666 to 3054
----- 523 remaining sequences to the next cycle
----- new table with 134 representatives
comparing sequences from 2531 to 3775
----- 681 remaining sequences to the next cycle
----- new table with 100 representatives
comparing sequences from 3094 to 4244
----- 598 remaining sequences to the next cycle
----- new table with 100 representatives
comparing sequences from 3646 to 4704
----- 140 remaining sequences to the next cycle
----- new table with 100 representatives
comparing sequences from 4564 to 5469
----- 347 remaining sequences to the next cycle
----- new table with 100 representatives
comparing sequences from 5122 to 5934
----- 328 remaining sequences to the next cycle
----- new table with 100 representatives
comparing sequences from 5606 to 6338
----- 258 remaining sequences to the next cycle
----- new table with 100 representatives
comparing sequences from 6080 to 6733
----- 207 remaining sequences to the next cycle
----- new table with 146 representatives
comparing sequences from 6526 to 7104

```

-----      115 remaining sequences to the next cycle
----- new table with      117 representatives
# comparing sequences from      6989 to      7490
...----- new table with      44 representatives
# comparing sequences from      7490 to      7908
99.9%----- new table with      1 representatives
# comparing sequences from      7908 to      8256
...----- new table with      28 representatives
# comparing sequences from      8256 to      8546
...----- new table with      48 representatives
# comparing sequences from      8546 to      8788
100.0%----- new table with      0 representatives
# comparing sequences from      8788 to      8989
----- new table with      0 representatives
# comparing sequences from      8989 to      9157
----- new table with      0 representatives
# comparing sequences from      9157 to      9999
----- new table with      194 representatives

```

9999 finished 1556 clusters

Approximated maximum memory consumption: 189M
writing new database
writing clustering information
program completed !

Total CPU time 3.08
CD-HIT concluído.

Executando CD-HIT...

```

=====
Program: CD-HIT, V4.8.1 (+OpenMP), Aug 20 2021, 08:39:56
Command: cd-hit -i pipelineW/negative.fasta -o
         pipelineW/negative_nr.fasta -c 0.7 -n 5 -M 30000 -T 4

```

Started: Wed Jun 10 09:31:31 2026

Output

```

-----
total seq: 9999
longest and shortest : 2993 and 100
Total letters: 4294300
Sequences have been sorted

```

Approximated minimal memory consumption:

Sequence	: 5M
Buffer	: 4 X 11M = 44M
Table	: 2 X 65M = 131M

Miscellaneous : 0M
Total : 181M

Table limit with the given memory limit:
Max number of representatives: 4000000
Max number of word counting entries: 3727287707

```
# comparing sequences from      0 to      1666
.----- new table with      1373 representatives
# comparing sequences from      1666 to      3054
----- 643 remaining sequences to the next cycle
----- new table with      619 representatives
# comparing sequences from      2411 to      3675
----- 801 remaining sequences to the next cycle
----- new table with      304 representatives
# comparing sequences from      2874 to      4061
----- 847 remaining sequences to the next cycle
----- new table with      237 representatives
# comparing sequences from      3214 to      4344
----- 771 remaining sequences to the next cycle
----- new table with      288 representatives
# comparing sequences from      3573 to      4644
----- 748 remaining sequences to the next cycle
----- new table with      273 representatives
# comparing sequences from      3896 to      4913
----- 712 remaining sequences to the next cycle
----- new table with      248 representatives
# comparing sequences from      4201 to      5167
----- 658 remaining sequences to the next cycle
----- new table with      232 representatives
# comparing sequences from      4509 to      5424
----- 625 remaining sequences to the next cycle
----- new table with      245 representatives
# comparing sequences from      4799 to      5665
----- 601 remaining sequences to the next cycle
----- new table with      194 representatives
# comparing sequences from      5064 to      5886
----- 544 remaining sequences to the next cycle
----- new table with      195 representatives
# comparing sequences from      5342 to      6118
----- 522 remaining sequences to the next cycle
----- new table with      184 representatives
# comparing sequences from      5596 to      6329
----- 512 remaining sequences to the next cycle
----- new table with      161 representatives
# comparing sequences from      5817 to      6514
----- 467 remaining sequences to the next cycle
----- new table with      163 representatives
```

```

# comparing sequences from      6047 to      6705
----- 447 remaining sequences to the next cycle
----- new table with      169 representatives
# comparing sequences from      6258 to      6881
----- 425 remaining sequences to the next cycle
----- new table with      126 representatives
# comparing sequences from      6456 to      7046
----- 404 remaining sequences to the next cycle
----- new table with      126 representatives
# comparing sequences from      6642 to      7201
----- 373 remaining sequences to the next cycle
----- new table with      141 representatives
# comparing sequences from      6828 to      7356
----- 335 remaining sequences to the next cycle
----- new table with      123 representatives
# comparing sequences from      7021 to      7517
----- 330 remaining sequences to the next cycle
----- new table with      131 representatives
# comparing sequences from      7187 to      7655
----- 288 remaining sequences to the next cycle
----- new table with      127 representatives
# comparing sequences from      7367 to      7805
----- 269 remaining sequences to the next cycle
----- new table with      104 representatives
# comparing sequences from      7536 to      7946
----- 255 remaining sequences to the next cycle
----- new table with      120 representatives
# comparing sequences from      7691 to      8075
----- 225 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from      7850 to      8208
----- 215 remaining sequences to the next cycle
----- new table with      105 representatives
# comparing sequences from      7993 to      8327
----- 192 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from      8135 to      8445
----- 158 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from      8287 to      8572
----- 140 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from      8432 to      8693
----- 95 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from      8598 to      8831
----- 23 remaining sequences to the next cycle
----- new table with      100 representatives

```

```

# comparing sequences from      8808  to      9006
----- 51 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from      8955  to      9129
----- 26 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from      9103  to      9999
...----- new table with      418 representatives

```

9999 finished 7206 clusters

```

Approximated maximum memory consumption: 194M
writing new database
writing clustering information
program completed !

```

Total CPU time 1.68
CD-HIT concluído.

Total de Features/Atributos: 440

Features/atributos:

```

['protein_length', 'AAC_A', 'AAC_C', 'AAC_D', 'AAC_E', 'AAC_F', 'AAC_G',
'AAC_H', 'AAC_I', 'AAC_K', 'AAC_L', 'AAC_M', 'AAC_N', 'AAC_P', 'AAC_Q', 'AAC_R',
'AAC_S', 'AAC_T', 'AAC_V', 'AAC_W', 'AAC_Y', 'DIPEP_AA', 'DIPEP_AC', 'DIPEP_AD',
'DIPEP_AE', 'DIPEP_AF', 'DIPEP_AG', 'DIPEP_AH', 'DIPEP_AI', 'DIPEP_AK',
'DIPEP_AL', 'DIPEP_AM', 'DIPEP_AN', 'DIPEP_AP', 'DIPEP_AQ', 'DIPEP_AR',
'DIPEP_AS', 'DIPEP_AT', 'DIPEP_AV', 'DIPEP_AW', 'DIPEP_AY', 'DIPEP_CA',
'DIPEP_CC', 'DIPEP_CD', 'DIPEP_CE', 'DIPEP_CF', 'DIPEP_CG', 'DIPEP_CH',
'DIPEP_CI', 'DIPEP_CK', 'DIPEP_CL', 'DIPEP_CM', 'DIPEP_CN', 'DIPEP_CP',
'DIPEP_CQ', 'DIPEP_CR', 'DIPEP_CS', 'DIPEP_CT', 'DIPEP_CV', 'DIPEP_CW',
'DIPEP_CY', 'DIPEP_DA', 'DIPEP_DC', 'DIPEP_DD', 'DIPEP_DE', 'DIPEP_DF',
'DIPEP_DG', 'DIPEP_DH', 'DIPEP_DI', 'DIPEP_DK', 'DIPEP_DL', 'DIPEP_DM',
'DIPEP_DN', 'DIPEP_DP', 'DIPEP_DQ', 'DIPEP_DR', 'DIPEP_DS', 'DIPEP_DT',
'DIPEP_DV', 'DIPEP_DW', 'DIPEP_DY', 'DIPEP_EA', 'DIPEP_EC', 'DIPEP_ED',
'DIPEP_EE', 'DIPEP_EF', 'DIPEP_EG', 'DIPEP_EH', 'DIPEP_EI', 'DIPEP_EK',
'DIPEP_EL', 'DIPEP_EM', 'DIPEP_EN', 'DIPEP_EP', 'DIPEP_EQ', 'DIPEP_ER',
'DIPEP_ES', 'DIPEP_ET', 'DIPEP_EV', 'DIPEP_EW', 'DIPEP_EY', 'DIPEP_FA',
'DIPEP_FC', 'DIPEP_FD', 'DIPEP_FE', 'DIPEP_FF', 'DIPEP_FG', 'DIPEP_FH',
'DIPEP_FI', 'DIPEP_FK', 'DIPEP_FL', 'DIPEP_FM', 'DIPEP_FN', 'DIPEP_FP',
'DIPEP_FQ', 'DIPEP_FR', 'DIPEP_FS', 'DIPEP_FT', 'DIPEP_FV', 'DIPEP_FW',
'DIPEP_FY', 'DIPEP_GA', 'DIPEP_GC', 'DIPEP_GD', 'DIPEP_GE', 'DIPEP_GF',
'DIPEP_GG', 'DIPEP_GH', 'DIPEP_GI', 'DIPEP_GK', 'DIPEP_GL', 'DIPEP_GM',
'DIPEP_GN', 'DIPEP_GP', 'DIPEP_GQ', 'DIPEP_GR', 'DIPEP_GS', 'DIPEP_GT',
'DIPEP_GV', 'DIPEP_GW', 'DIPEP_GY', 'DIPEP_HA', 'DIPEP_HC', 'DIPEP_HD',
'DIPEP_HE', 'DIPEP_HF', 'DIPEP_HG', 'DIPEP_HH', 'DIPEP_HI', 'DIPEP_HK',
'DIPEP_HL', 'DIPEP_HM', 'DIPEP_HN', 'DIPEP_HP', 'DIPEP_HQ', 'DIPEP_HR',
'DIPEP_HS', 'DIPEP_HT', 'DIPEP_HV', 'DIPEP_HW', 'DIPEP_HY', 'DIPEP_IA',

```

'DIPEP_IC', 'DIPEP_ID', 'DIPEP_IE', 'DIPEP_IF', 'DIPEP_IG', 'DIPEP_IH',
 'DIPEP_II', 'DIPEP_IK', 'DIPEP_IL', 'DIPEP_IM', 'DIPEP_IN', 'DIPEP_IP',
 'DIPEP_IQ', 'DIPEP_IR', 'DIPEP_IS', 'DIPEP_IT', 'DIPEP_IV', 'DIPEP_IW',
 'DIPEP_IY', 'DIPEP_KA', 'DIPEP_KC', 'DIPEP_KD', 'DIPEP_KE', 'DIPEP_KF',
 'DIPEP_KG', 'DIPEP_KH', 'DIPEP_KI', 'DIPEP_KK', 'DIPEP_KL', 'DIPEP_KM',
 'DIPEP_KN', 'DIPEP_KP', 'DIPEP_KQ', 'DIPEP_KR', 'DIPEP_KS', 'DIPEP_KT',
 'DIPEP_KV', 'DIPEP_KW', 'DIPEP_KY', 'DIPEP_LA', 'DIPEP_LC', 'DIPEP_LD',
 'DIPEP_LE', 'DIPEP_LF', 'DIPEP_LG', 'DIPEP_LH', 'DIPEP_LI', 'DIPEP_LK',
 'DIPEP_LL', 'DIPEP_LM', 'DIPEP_LN', 'DIPEP_LP', 'DIPEP_LQ', 'DIPEP_LR',
 'DIPEP_LS', 'DIPEP_LT', 'DIPEP_LV', 'DIPEP_LW', 'DIPEP_LY', 'DIPEP_MA',
 'DIPEP_MC', 'DIPEP_MD', 'DIPEP_ME', 'DIPEP_MF', 'DIPEP_MG', 'DIPEP_MH',
 'DIPEP_MI', 'DIPEP_MK', 'DIPEP_ML', 'DIPEP_MM', 'DIPEP_MN', 'DIPEP_MP',
 'DIPEP_MQ', 'DIPEP_MR', 'DIPEP_MS', 'DIPEP_MT', 'DIPEP_MV', 'DIPEP_MW',
 'DIPEP_MY', 'DIPEP_NA', 'DIPEP_NC', 'DIPEP_ND', 'DIPEP_NE', 'DIPEP_NF',
 'DIPEP_NG', 'DIPEP_NH', 'DIPEP_NI', 'DIPEP_NK', 'DIPEP_NL', 'DIPEP_NM',
 'DIPEP_NN', 'DIPEP_NP', 'DIPEP_NQ', 'DIPEP_NR', 'DIPEP_NS', 'DIPEP_NT',
 'DIPEP_NV', 'DIPEP_NW', 'DIPEP_NY', 'DIPEP_PA', 'DIPEP_PC', 'DIPEP_PD',
 'DIPEP_PE', 'DIPEP_PF', 'DIPEP_PG', 'DIPEP_PH', 'DIPEP_PI', 'DIPEP_PK',
 'DIPEP_PL', 'DIPEP_PM', 'DIPEP_PN', 'DIPEP_PP', 'DIPEP_PQ', 'DIPEP_PR',
 'DIPEP_PS', 'DIPEP_PT', 'DIPEP_PV', 'DIPEP_PW', 'DIPEP_PY', 'DIPEP_QA',
 'DIPEP_QC', 'DIPEP_QD', 'DIPEP_QE', 'DIPEP_QF', 'DIPEP_QG', 'DIPEP_QH',
 'DIPEP_QI', 'DIPEP_QK', 'DIPEP_QL', 'DIPEP_QM', 'DIPEP_QN', 'DIPEP_QP',
 'DIPEP_QQ', 'DIPEP_QR', 'DIPEP_QS', 'DIPEP_QT', 'DIPEP_QV', 'DIPEP_QW',
 'DIPEP_QY', 'DIPEP_RA', 'DIPEP_RC', 'DIPEP_RD', 'DIPEP_RE', 'DIPEP_RF',
 'DIPEP_RG', 'DIPEP_RH', 'DIPEP_RI', 'DIPEP_RK', 'DIPEP_RL', 'DIPEP_RM',
 'DIPEP_RN', 'DIPEP_RP', 'DIPEP_RQ', 'DIPEP_RR', 'DIPEP_RS', 'DIPEP_RT',
 'DIPEP_RV', 'DIPEP_RW', 'DIPEP_RY', 'DIPEP_SA', 'DIPEP_SC', 'DIPEP_SD',
 'DIPEP_SE', 'DIPEP_SF', 'DIPEP_SG', 'DIPEP_SH', 'DIPEP_SI', 'DIPEP_SK',
 'DIPEP_SL', 'DIPEP_SM', 'DIPEP_SN', 'DIPEP_SP', 'DIPEP_SQ', 'DIPEP_SR',
 'DIPEP_SS', 'DIPEP_ST', 'DIPEP_SV', 'DIPEP_SW', 'DIPEP_SY', 'DIPEP_TA',
 'DIPEP_TC', 'DIPEP_TD', 'DIPEP_TE', 'DIPEP_TF', 'DIPEP_TG', 'DIPEP_TH',
 'DIPEP_TI', 'DIPEP_TK', 'DIPEP_TL', 'DIPEP_TM', 'DIPEP_TN', 'DIPEP_TP',
 'DIPEP_TQ', 'DIPEP_TR', 'DIPEP_TS', 'DIPEP_TT', 'DIPEP_TV', 'DIPEP_TW',
 'DIPEP_TY', 'DIPEP_VA', 'DIPEP_VC', 'DIPEP_VD', 'DIPEP_VE', 'DIPEP_VF',
 'DIPEP_VG', 'DIPEP_VH', 'DIPEP_VI', 'DIPEP_VK', 'DIPEP_VL', 'DIPEP_VM',
 'DIPEP_VN', 'DIPEP_VP', 'DIPEP_VQ', 'DIPEP_VR', 'DIPEP_VS', 'DIPEP_VT',
 'DIPEP_VV', 'DIPEP_VW', 'DIPEP_VY', 'DIPEP_WA', 'DIPEP_WC', 'DIPEP_WD',
 'DIPEP_WE', 'DIPEP_WF', 'DIPEP_WG', 'DIPEP_WH', 'DIPEP_WI', 'DIPEP_WK',
 'DIPEP_WL', 'DIPEP_WM', 'DIPEP_WN', 'DIPEP_WP', 'DIPEP_WQ', 'DIPEP_WR',
 'DIPEP_WS', 'DIPEP_WT', 'DIPEP_WV', 'DIPEP_WW', 'DIPEP_WY', 'DIPEP_YA',
 'DIPEP_YC', 'DIPEP_YD', 'DIPEP_YE', 'DIPEP_YF', 'DIPEP_YG', 'DIPEP_YH',
 'DIPEP_YI', 'DIPEP_YK', 'DIPEP_YL', 'DIPEP_YM', 'DIPEP_YN', 'DIPEP_YP',
 'DIPEP_YQ', 'DIPEP_YR', 'DIPEP_YS', 'DIPEP_YT', 'DIPEP_YV', 'DIPEP_YW',
 'DIPEP_YY', 'P_LOOP', 'KINASE2', 'RNBS_A', 'RNBS_B', 'RNBS_C', 'GLPL', 'RNBS_D',
 'MHD', 'num_motifs_detectados', 'dist_ploop_kinase2', 'dist_kinase2_glpl',
 'dist_glpl_mhd', 'dist_ploop_kinase2_norm', 'dist_kinase2_glpl_norm',
 'dist_glpl_mhd_norm', 'has_ploop', 'has_kinase2', 'has_glpl', 'has_mhd']

Clusters ausentes:
125

Dataset original pós CD-HIT: (8625, 443)

--- Executando Balanceamento de Classes ---

Contagem inicial -> Positivos (R-genes): 1511 | Negativos (Background): 7114

Subamostragem aplicada com sucesso nos Negativos.

Contagem final -> Positivos: 1511 | Negativos: 1511

Dataset Final Pronto: (3022, 443)

Fim tratamento dos dados...

```
[6]: '''  
      k-Fold-Cross-Validation  
      '''  
  
def validacao_cruzada(nome, model, X_train, y_train, groups_train):  
  
    cv = StratifiedGroupKFold(  
        n_splits=5, #10  
        shuffle=True,  
        random_state=42  
    )  
    mcc_scorer = make_scorer(matthews_corrcoef)  
    scoring_dict = {  
        'recall': 'recall',  
        'roc_auc': 'roc_auc',  
        'mcc': mcc_scorer,  
        'accuracy': 'accuracy',  
        'precision': 'precision',  
        'f1': 'f1'  
    }  
    scores = cross_validate(  
        model,  
        X_train,  
        y_train,  
        groups=groups_train,  
        cv=cv,  
        scoring=scoring_dict,  
        n_jobs=-1  
    )  
    print(f"Finalizada a validação cruzada para: {nome}")  
    return scores
```

```
[7]: '''
      Criação dos modelos/ Treinamento
      '''

      # para melhor separar os conjuntos de treino e teste considerando os
      # agrupamentos feitos no CD-HIT para sequências muito parecidas
      def split_por_homologia(df):
          print(df.head(30))
          groups = df["cluster"]
          splitter = GroupShuffleSplit(
              n_splits=1,
              test_size=0.2,
              random_state=42
          )
          # cv = StratifiedGroupKFold(
          #     n_splits = 10,
          #     shuffle=True,
          #     random_state = 42
          # )

          train_idx, test_idx = next(
              splitter.split(df, groups=groups)
              # cv.split(df, df["target"], groups)
          )
          train_df = df.iloc[train_idx]
          test_df = df.iloc[test_idx]
          return train_df, test_df

      def calcular_e_salvar_shap(nome, model, X_test, feature_names, output_dir):
          """
          Calcula os SHAP values para modelos de árvore e salva os gráficos
          explicativos.
          """
          print(f"\n[SHAP] Inicializando análise de interpretabilidade para: {nome}...")

          # 1. Inicializa o explicador otimizado para árvores
          explainer = shap.TreeExplainer(model)

          # 2. Calcula os SHAP values para o conjunto de teste
          shap_values = explainer.shap_values(X_test)

          # 3. Tratamento de formato (Scikit-Learn Random Forest vs XGBoost)
          # O Random Forest do sklearn retorna uma lista contendo [valores_classe_0,
          # valores_classe_1]
          # O XGBoost retorna diretamente a matriz de SHAP values para a classe alvo
          if isinstance(shap_values, list):
```

```

        # Selecionamos o índice 1 (classe positiva: proteínas de resistência/
        ↪alvos)
        shap_values_pos = shap_values[1]
        elif isinstance(shap_values, np.ndarray) and len(shap_values.shape) == 3:
            # Tratamento para versões específicas do SHAP que geram arrays 3D
            shap_values_pos = shap_values[:, :, 1]
        else:
            shap_values_pos = shap_values

    # 4. Gráfico 1: Summary Plot (Beeswarm)
    # Mostra o impacto biológico de cada feature (ex: se alta frequência
    ↪aumenta ou diminui a predição)
    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values_pos,
        X_test,
        feature_names=feature_names,
        max_display=20, # Foco nas top 20 características
        show=False
    )
    # plt.title(f"SHAP Summary Plot (Top 20) - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_summary.png", bbox_inches="tight",
    ↪dpi=300)
    plt.close()

    # 5. Gráfico 2: Bar Plot (Importância Global)
    # Mostra a magnitude média do impacto de cada feature de forma absoluta
    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values_pos,
        X_test,
        feature_names=feature_names,
        plot_type="bar",
        max_display=20,
        show=False
    )
    # plt.title(f"SHAP Global Feature Importance - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_bar.png", bbox_inches="tight",
    ↪dpi=300)
    plt.close()

    print(f"[SHAP] Gráficos salvos com sucesso em '{output_dir}/'!")

def calcular_shap_svm(nome, model, X_test, y_test, output_dir):

```

```

# =====
# IMPORTÂNCIA POR PERMUTAÇÃO PARA O SVM (Novo)
# =====
print("\n[SVM] Calculando a importância por permutação dos atributos...")

# Calcula a permutação no X_test (pode usar X_train se quiser focar no
↪ajuste ao treino)
result_svm = permutation_importance(model, X_test, y_test, n_repeats=5,
↪random_state=42, n_jobs=-1)

# Monta o DataFrame com os resultados
importancia_svm = pd.DataFrame({
    'feature': columns,
    'importance_mean': result_svm.importances_mean
}).sort_values('importance_mean', ascending=False)

print("\nTop 20 features mais importantes para o SVM:")
print(importancia_svm.head(20))

# Gera o gráfico de barras igual ao do XGBoost
top_svm = importancia_svm.head(20)
plt.figure(figsize=(10,8))
plt.barh(top_svm["feature"][:-1], top_svm["importance_mean"][:-1],
↪color='seagreen')
plt.xlabel("Queda no Desempenho (Média)")
plt.ylabel("Feature / Atributo")
# plt.title("Top 20 Features - SVM Permutation Importance")
plt.tight_layout()
plt.savefig(f"{output_dir}/svm_importancia_permutacao.png",
↪bbox_inches="tight", dpi=300)
plt.close()
print(f"[SVM] Gráfico salvo com sucesso em '{output_dir}/"
↪svm_importancia_permutacao.png'!")

# =====
# TREINAMENTO - treina os modelos
# =====

def avaliar_modelo(nome, model, X_train, y_train, X_test, y_test):
    print("\n=====")
    print(nome)
    print("=====")
    model.fit(X_train, y_train)

    probs = model.predict_proba(X_test)[:,-1]
    preds = (probs >= 0.5).astype(int)
    print(classification_report(y_test, preds))

```



```

roc = roc_auc_score(y_test, probs)
pr = average_precision_score(y_test, probs)
mcc = matthews_corrcoef(y_test, preds)
f1 = f1_score(y_test, preds)
precision = precision_score(y_test, preds)
recall = recall_score(y_test, preds)
bal_acc = balanced_accuracy_score(y_test, preds)

print("ROC-AUC          :", roc)
print("PR-AUC          :", pr)
print("MCC             :", mcc)
print("F1              :", f1)
print("Precision        :", precision)
print("Recall           :", recall)
print("Balanced Accuracy:", bal_acc)

# =====
# MATRIZ DE CONFUSÃO
# =====
cm = confusion_matrix(y_test, preds)
plt.figure(figsize=(5,5))
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Negativo", "Positivo"]
)
disp.plot(cmap="Blues")
# plt.title(f"Matriz de Confusão - {nome}")
plt.savefig(
    f"{OUTPUT_DIR}/{nome}_matriz_confusao.png",
    bbox_inches="tight"
)
plt.close()

# =====
# CURVA ROC
# =====
fpr, tpr, _ = roc_curve(y_test, probs)
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(6,6))
plt.plot(
    fpr,
    tpr,
    label=f"AUC = {roc_auc:.4f}"
)
plt.plot([0,1], [0,1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
# plt.title(f"Curva ROC - {nome}")

```

```

plt.legend(loc="lower right")
plt.savefig(
    f"{OUTPUT_DIR}/{nome}_roc.png",
    bbox_inches="tight"
)
plt.close()

return {
    "Modelo": nome,
    "ROC_AUC": roc,
    "F1": f1,
    "Precision": precision,
    "Recall": recall,
    "Balanced_Accuracy": bal_acc,
    "MCC": mcc
}, model

# =====
# Separação Treino/Teste 80/20
# SPLIT POR HOMOLOGIA
# =====

train_df, test_df = split_por_homologia(df)
X_train = train_df[columns]
y_train = train_df["target"]
X_test = test_df[columns]
y_test = test_df["target"]
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# MODELOS
# =====

rf = RandomForestClassifier(
    n_estimators=1000,
    max_depth = 12,
    min_samples_split=4,
    max_features='sqrt',
    random_state=42,
    class_weight="balanced",
    n_jobs = -1
)
xgb = XGBClassifier(
    n_estimators=1200,

```

```

        max_depth=5,
        learning_rate=0.01,
        subsample=0.8,
        colsample_bytree=0.8,
        eval_metric="logloss",
        random_state=42,
        min_child_weight=3,
        reg_alpha=0.5,
        reg_lambda=2,
        scale_pos_weight=1.2
    )
    svm = SVC(
        probability=True,
        kernel="rbf",
        class_weight="balanced",
        random_state=42,
        C=2,
        gamma='scale'
    )
    scores = []
    groups_train = train_df["cluster"]

    scores_rf = validacao_cruzada('RF', rf, X_train, y_train, groups_train)
    scores_xgb = validacao_cruzada('XGB', xgb, X_train, y_train, groups_train)
    scores_svm = validacao_cruzada('SVM', svm, X_train, y_train, groups_train)

    print("Cross-Validation:RF")
    for k,v in scores_rf.items():
        if "test" in k:
            print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

    print("Cross-Validation XGB")
    for k,v in scores_xgb.items():
        if "test" in k:
            print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

    print("Cross-Validation SVM")
    for k,v in scores_svm.items():
        if "test" in k:
            print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

    resultados_cv = pd.DataFrame({
        'Modelo': ['RF', 'XGB', 'SVM'],
        'Accuracy': [
            scores_rf['test_accuracy'].mean(),
            scores_xgb['test_accuracy'].mean(),

```

```

        scores_svm['test_accuracy'].mean(),
    ],
    'Recall': [
        scores_rf['test_recall'].mean(),
        scores_xgb['test_recall'].mean(),
        scores_svm['test_recall'].mean(),
    ],
    'Precision': [
        scores_rf['test_precision'].mean(),
        scores_xgb['test_precision'].mean(),
        scores_svm['test_precision'].mean(),
    ],
    'F1': [
        scores_rf['test_f1'].mean(),
        scores_xgb['test_f1'].mean(),
        scores_svm['test_f1'].mean(),
    ],
    'ROC_AUC': [
        scores_rf['test_roc_auc'].mean(),
        scores_xgb['test_roc_auc'].mean(),
        scores_svm['test_roc_auc'].mean(),
    ],
    'MCC': [
        scores_rf['test_mcc'].mean(),
        scores_xgb['test_mcc'].mean(),
        scores_svm['test_mcc'].mean(),
    ]
})
print(resultados_cv)
resultados_cv.to_csv(
    f"{OUTPUT_DIR}/metricas_cv.csv",
    index=False
)
datos = []

for score in scores_rf['test_roc_auc']:
    datos.append(['RF', score])
for score in scores_xgb['test_roc_auc']:
    datos.append(['XGB', score])
for score in scores_svm['test_roc_auc']:
    datos.append(['SVM', score])

grafico = pd.DataFrame(datos, columns=['Modelo', 'ROC_AUC'])
grafico.boxplot(by='Modelo')
plt.ylabel("ROC-AUC")
# plt.title("10-Fold Cross Validation")
plt.suptitle("")

```

```

plt.tight_layout()
# plt.show()
plt.savefig(
    f"{OUTPUT_DIR}/kfold_cross_validation_roc.png",
    bbox_inches="tight"
)
plt.close()

#####
# Treinamento

results = []
r1, rf_model = avaliar_modelo(
    "Random Forest",
    rf,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r1)
calcular_e_salvar_shap("Random Forest", rf_model, X_test, columns, OUTPUT_DIR )

r2, xgb_model = avaliar_modelo(
    "XGBoost",
    xgb,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r2)
calcular_e_salvar_shap("XGBoost", xgb_model, X_test, columns, OUTPUT_DIR )

r3, svm_model = avaliar_modelo(
    "SVM",
    svm,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r3)
calcular_shap_svm("SVM", svm_model, X_test, y_test, OUTPUT_DIR)

```

```
print("\nFim criação/treino dos modelos\n")
```

	protein_length	AAC_A	AAC_C	AAC_D	AAC_E	AAC_F	\
0	1053	0.034188	0.018993	0.043685	0.038936	0.058879	
1	231	0.090909	0.012987	0.069264	0.077922	0.051948	
2	201	0.039801	0.004975	0.034826	0.044776	0.019900	
3	888	0.055180	0.018018	0.061937	0.072072	0.045045	
4	504	0.103175	0.019841	0.045635	0.071429	0.029762	
5	704	0.049716	0.007102	0.080966	0.117898	0.041193	
6	535	0.095327	0.009346	0.020561	0.026168	0.082243	
7	450	0.040000	0.028889	0.035556	0.082222	0.024444	
8	971	0.045314	0.017508	0.054583	0.071061	0.036045	
9	666	0.039039	0.016517	0.057057	0.057057	0.040541	
10	929	0.034446	0.034446	0.046286	0.069968	0.053821	
11	709	0.053597	0.014104	0.049365	0.071932	0.043724	
12	999	0.053053	0.018018	0.070070	0.050050	0.036036	
13	1071	0.056956	0.020542	0.059757	0.038282	0.043884	
14	1050	0.043810	0.022857	0.053333	0.080952	0.041905	
15	507	0.067061	0.023669	0.047337	0.055227	0.063116	
16	904	0.042035	0.022124	0.032080	0.055310	0.056416	
17	1152	0.065972	0.015625	0.041667	0.046875	0.030382	
18	949	0.033720	0.034773	0.052687	0.067439	0.041096	
19	272	0.055147	0.033088	0.040441	0.084559	0.040441	
20	428	0.049065	0.023364	0.070093	0.070093	0.051402	
21	253	0.059289	0.019763	0.071146	0.051383	0.039526	
22	886	0.038375	0.024831	0.060948	0.055305	0.050790	
23	910	0.052747	0.017582	0.060440	0.065934	0.052747	
24	239	0.041841	0.020921	0.083682	0.050209	0.046025	
25	947	0.039071	0.023231	0.062302	0.065470	0.047518	
26	126	0.055556	0.031746	0.055556	0.055556	0.039683	
27	104	0.028846	0.038462	0.038462	0.067308	0.028846	
28	105	0.057143	0.019048	0.076190	0.019048	0.057143	
29	827	0.044740	0.022975	0.072551	0.073761	0.044740	

	AAC_G	AAC_H	AAC_I	AAC_K	...	dist_ploop_kinase2_norm	\
0	0.059829	0.017094	0.056980	0.036087	...	-1.000000	
1	0.077922	0.038961	0.043290	0.082251	...	-1.000000	
2	0.054726	0.024876	0.074627	0.084577	...	-1.000000	
3	0.047297	0.023649	0.061937	0.085586	...	0.087838	
4	0.081349	0.005952	0.075397	0.073413	...	-1.000000	
5	0.048295	0.009943	0.065341	0.117898	...	-1.000000	
6	0.106542	0.014953	0.065421	0.039252	...	-1.000000	
7	0.048889	0.006667	0.075556	0.091111	...	-1.000000	
8	0.055613	0.024717	0.057673	0.061792	...	-1.000000	
9	0.052553	0.018018	0.078078	0.064565	...	-1.000000	
10	0.051668	0.027987	0.071044	0.059203	...	0.081808	
11	0.063470	0.021157	0.063470	0.077574	...	-1.000000	
12	0.069069	0.021021	0.043043	0.077077	...	-1.000000	

13	0.090570	0.023343	0.040149	0.059757	...	-1.000000
14	0.048571	0.024762	0.065714	0.058095	...	0.071429
15	0.047337	0.027613	0.061144	0.076923	...	-1.000000
16	0.059735	0.028761	0.065265	0.074115	...	-1.000000
17	0.110243	0.026910	0.039062	0.046007	...	-1.000000
18	0.049526	0.025290	0.060063	0.052687	...	0.079031
19	0.044118	0.029412	0.047794	0.066176	...	-1.000000
20	0.053738	0.011682	0.072430	0.077103	...	-1.000000
21	0.063241	0.027668	0.075099	0.102767	...	0.272727
22	0.053047	0.027088	0.067720	0.082393	...	0.086907
23	0.040659	0.021978	0.071429	0.086813	...	0.087912
24	0.033473	0.025105	0.092050	0.079498	...	-1.000000
25	0.043295	0.032735	0.076030	0.059134	...	0.083421
26	0.055556	0.039683	0.079365	0.095238	...	-1.000000
27	0.067308	0.019231	0.067308	0.076923	...	-1.000000
28	0.057143	0.019048	0.066667	0.076190	...	-1.000000
29	0.041112	0.018138	0.062878	0.059250	...	0.097944

	dist_kinase2_glpl_norm	dist_glpl_mhd_norm	has_ploop	has_kinase2	\
0	-1.000000	-1.00	0	0	
1	-1.000000	-1.00	0	0	
2	-1.000000	-1.00	0	0	
3	0.097973	-1.00	1	1	
4	-1.000000	-1.00	0	0	
5	-1.000000	-1.00	0	0	
6	-1.000000	-1.00	0	0	
7	-1.000000	-1.00	0	0	
8	-1.000000	-1.00	1	0	
9	-1.000000	-1.00	0	0	
10	-1.000000	-1.00	1	1	
11	-1.000000	-1.00	0	0	
12	-1.000000	-1.00	0	0	
13	-1.000000	-1.00	0	0	
14	0.083810	0.12	1	1	
15	-1.000000	-1.00	0	0	
16	-1.000000	-1.00	0	0	
17	-1.000000	-1.00	0	0	
18	0.088514	-1.00	1	1	
19	-1.000000	-1.00	0	0	
20	-1.000000	-1.00	1	0	
21	-1.000000	-1.00	1	1	
22	0.104966	-1.00	1	1	
23	-1.000000	-1.00	1	1	
24	-1.000000	-1.00	1	0	
25	0.093981	-1.00	1	1	
26	-1.000000	-1.00	0	0	
27	-1.000000	-1.00	0	0	
28	-1.000000	-1.00	0	0	

29 -1.000000 -1.00 1 1

	has_gipl	has_mhd	target	id	cluster
0	1	0	1	NP_001234474.3	POS_388
1	0	0	1	NP_001234459.3	POS_1294
2	0	0	1	YEV57985.1	POS_1361
3	1	0	1	NP_001234202.1	POS_621
4	0	0	1	NP_001333606.1	POS_1029
5	0	0	1	NP_001308492.1	POS_863
6	0	0	1	NP_001307892.1	POS_1007
7	0	0	1	NP_001234240.1	POS_1072
8	1	0	1	CAQ5402917.1	POS_472
9	0	0	1	CAQ5402962.1	POS_903
10	0	0	1	CAQ5404663.1	NEG_494
11	1	0	1	CAQ5406153.1	POS_858
12	0	0	1	CAQ5404100.1	POS_435
13	0	0	1	CAQ5404098.1	POS_370
14	1	1	1	CAQ5403806.1	NEG_321
15	0	0	1	CAQ5403796.1	POS_1026
16	0	0	1	CAQ5409432.1	POS_584
17	0	0	1	CAQ5408022.1	POS_308
18	1	0	1	CAQ5407207.1	POS_498
19	1	0	1	CAQ5411306.1	POS_1229
20	1	0	1	CAQ5411102.1	POS_1086
21	0	0	1	CAQ5410995.1	POS_1255
22	1	0	1	CAQ5413609.1	POS_625
23	0	0	1	CAQ5413612.1	POS_569
24	0	0	1	CAQ5410852.1	POS_1281
25	1	0	1	CAQ5410851.1	POS_500
26	0	0	1	CAQ5413789.1	POS_1501
27	1	0	1	CAQ5410659.1	POS_1520
28	0	0	1	CAQ5410656.1	POS_1519
29	0	1	1	CAQ5410655.1	POS_728

[30 rows x 443 columns]

Finalizada a validação cruzada para: RF

Finalizada a validação cruzada para: XGB

Finalizada a validação cruzada para: SVM

Cross-Validation:RF

test_recall: 0.7423 ± 0.0392

test_roc_auc: 0.9321 ± 0.0047

test_mcc: 0.7386 ± 0.0383

test_accuracy: 0.8581 ± 0.0219

test_precision: 0.9710 ± 0.0098

test_f1: 0.8409 ± 0.0276

Cross-Validation XGB

test_recall: 0.8304 ± 0.0335

test_roc_auc: 0.9391 ± 0.0060


```

test_mcc: 0.7683 ± 0.0142
test_accuracy: 0.8813 ± 0.0088
test_precision: 0.9291 ± 0.0197
test_f1: 0.8762 ± 0.0119
Cross-Validation SVM
test_recall: 0.8108 ± 0.0345
test_roc_auc: 0.9436 ± 0.0099
test_mcc: 0.7774 ± 0.0257
test_accuracy: 0.8837 ± 0.0151
test_precision: 0.9532 ± 0.0067
test_f1: 0.8758 ± 0.0185

```

	Modelo	Accuracy	Recall	Precision	F1	ROC_AUC	MCC
0	RF	0.858090	0.742286	0.970984	0.840904	0.932073	0.738640
1	XGB	0.881254	0.830357	0.929112	0.876177	0.939079	0.768331
2	SVM	0.883739	0.810788	0.953240	0.875779	0.943598	0.777404

```

=====
Random Forest
=====

```

	precision	recall	f1-score	support
0	0.81	0.97	0.88	320
1	0.95	0.74	0.83	285
accuracy			0.86	605
macro avg	0.88	0.85	0.86	605
weighted avg	0.88	0.86	0.86	605

```

ROC-AUC      : 0.9292653508771929
PR-AUC       : 0.9428836988366597
MCC          : 0.7350953791456244
F1           : 0.83399209486166
Precision    : 0.9547511312217195
Recall       : 0.7403508771929824
Balanced Accuracy: 0.8545504385964913

```

```

[SHAP] Inicializando análise de interpretabilidade para: Random Forest...
[SHAP] Gráficos salvos com sucesso em 'pipelineW/'!

```

```

=====
XGBoost
=====

```

	precision	recall	f1-score	support
0	0.85	0.94	0.89	320
1	0.92	0.82	0.87	285
accuracy			0.88	605

macro avg	0.89	0.88	0.88	605
weighted avg	0.89	0.88	0.88	605

ROC-AUC : 0.9455811403508771
 PR-AUC : 0.953695932086554
 MCC : 0.7672072919872929
 F1 : 0.8682745825602969
 Precision : 0.9212598425196851
 Recall : 0.8210526315789474
 Balanced Accuracy: 0.8792763157894736

[SHAP] Inicializando análise de interpretabilidade para: XGBoost...
 [SHAP] Gráficos salvos com sucesso em 'pipelineW/'!

SVM

	precision	recall	f1-score	support
0	0.88	0.93	0.90	320
1	0.92	0.85	0.89	285
accuracy			0.90	605
macro avg	0.90	0.89	0.89	605
weighted avg	0.90	0.90	0.90	605

ROC-AUC : 0.9453618421052632
 PR-AUC : 0.9545406208657615
 MCC : 0.7921303057356509
 F1 : 0.8852459016393442
 Precision : 0.9204545454545454
 Recall : 0.8526315789473684
 Balanced Accuracy: 0.8935032894736842

[SVM] Calculando a importância por permutação dos atributos...

Top 20 features mais importantes para o SVM:

	feature	importance_mean
312	DIPEP_RN	2.975207e-03
101	DIPEP_FA	1.322314e-03
248	DIPEP_NI	1.322314e-03
261	DIPEP_PA	6.611570e-04
124	DIPEP_GE	6.611570e-04
19	AAC_W	3.305785e-04
230	DIPEP_ML	3.305785e-04
99	DIPEP_EW	3.305785e-04
34	DIPEP_AQ	3.305785e-04
415	DIPEP_YR	4.440892e-17

```

330          DIPEP_SL      4.440892e-17
251          DIPEP_NM      2.220446e-17
217          DIPEP_LT      2.220446e-17
111          DIPEP_FM      2.220446e-17
92           DIPEP_EN      2.220446e-17
313          DIPEP_RP      2.220446e-17
352          DIPEP_TN      0.000000e+00
367          DIPEP_VH      0.000000e+00
434  dist_kinase2_glp1_norm  0.000000e+00
392          DIPEP_WN      0.000000e+00
[SVM] Gráfico salvo com sucesso em 'pipelineW/svm_importancia_permutacao.png'!

```

Fim criação/treino dos modelos

<Figure size 500x500 with 0 Axes>

<Figure size 500x500 with 0 Axes>

<Figure size 500x500 with 0 Axes>

```

[8]: '''
    Avaliação e Validação dos modelos
    '''

    # verificar importância dos atributos/feature para o Random Forest
    importancia = pd.DataFrame({"feature":columns,"importance":rf_model.
        ↳feature_importances_})
    importancia = importancia.sort_values("importance",ascending = False)
    print("\nTop 20 features: Random Forest")
    print(importancia.head(20))
    # GRÁFICO
    top = importancia.head(20)
    plt.figure(figsize=(10,8))
    plt.barh( top["feature"][::-1], top["importance"][::-1] )
    plt.xlabel("Importância")
    plt.ylabel("Feature")
    # plt.title("Top 20 Features - Random Forest")
    plt.tight_layout()
    plt.savefig(
        f"{OUTPUT_DIR}/rf_importancia_variaveis.png",
        bbox_inches="tight"
    )
    plt.close()

    # verificar importância dos atributos/feature para o XGBoost
    importancia = pd.DataFrame({"feature":columns, "importance":xgb_model.
        ↳feature_importances_})
    importancia = importancia.sort_values("importance", ascending = False)
    print("\nTop 20 features: XGBoost")

```

```

print(importancia.head(20))
# GRÁFICO
top = importancia.head(20)
plt.figure(figsize=(10,8))
plt.barh(top["feature"][::-1], top["importance"][::-1])
plt.xlabel("Importância")
plt.ylabel("Feature")
# plt.title("Top 20 Features - XGBoost")
plt.tight_layout()
plt.savefig(
    f"{OUTPUT_DIR}/xgb_importancia_variaveis.png",
    bbox_inches="tight"
)
plt.close()

print("\nCOMPARAÇÃO MODELOS")
print(pd.DataFrame(results))

results_df = pd.DataFrame(results)
results_df.to_csv(
    f"{OUTPUT_DIR}/metricas_comparacao.csv",
    index=False
)

metrics = [
    "ROC_AUC",
    "F1",
    "Precision",
    "Recall",
    "Balanced_Accuracy"
]

for metric in metrics:
    plt.figure(figsize=(8,5))
    plt.bar(
        results_df["Modelo"],
        results_df[metric]
    )

    plt.ylim(0, 1)
    plt.ylabel(metric)
    # plt.title(f"Comparação dos Modelos - {metric}")
    for i, v in enumerate(results_df[metric]):
        plt.text(i, v + 0.01, f"{v:.3f}", ha='center')

    plt.savefig(
        f"{OUTPUT_DIR}/comparacao_{metric}.png",

```

```

        bbox_inches="tight"
    )
    plt.close()

modelos = {
    "Random Forest": rf_model,
    "XGBoost": xgb_model,
    "SVM": svm_model
}

plt.figure(figsize=(7,7))
for nome, model in modelos.items():
    probs = model.predict_proba(X_test)[: ,1]
    fpr, tpr, _ = roc_curve(y_test, probs)
    roc_auc = auc(fpr, tpr)
    plt.plot(
        fpr,
        tpr,
        label=f"{nome} AUC={roc_auc:.3f}"
    )
plt.plot([0,1],[0,1], '--')
plt.xlabel("FPR")
plt.ylabel("TPR")
# plt.title("Comparação ROC")
plt.legend()
plt.savefig(f"{OUTPUT_DIR}/roc_comparativa.png")
plt.close()

print("Fim avaliação / validação modelos ")

```

Top 20 features: Random Forest

	feature	importance
10	AAC_L	0.034045
429	num_motifs_detectados	0.030923
421	P_LOOP	0.026130
437	has_kinase2	0.025599
436	has_ploop	0.024851
1	AAC_A	0.024333
438	has_glp1	0.022571
426	GLPL	0.022070
209	DIPEP_LK	0.021539
422	KINASE2	0.019401
204	DIPEP_LE	0.015553
433	dist_ploop_kinase2_norm	0.015400
430	dist_ploop_kinase2	0.014890
250	DIPEP_NL	0.014546
0	protein_length	0.014026
190	DIPEP_KL	0.010364

150	DIPEP_HL	0.009666
13	AAC_P	0.009355
431	dist_kinase2_glp1	0.008857
427	RNBS_D	0.008818

Top 20 features: XGBoost

	feature	importance
421	P_LOOP	0.107363
436	has_ploop	0.060549
429	num_motifs_detectados	0.054139
438	has_glp1	0.028675
426	GLPL	0.021303
422	KINASE2	0.019446
437	has_kinase2	0.019148
430	dist_ploop_kinase2	0.009783
1	AAC_A	0.009699
10	AAC_L	0.009344
0	protein_length	0.006025
209	DIPEP_LK	0.004882
427	RNBS_D	0.004295
106	DIPEP_FG	0.003932
15	AAC_R	0.003692
333	DIPEP_SP	0.003574
341	DIPEP_TA	0.003456
146	DIPEP_HG	0.003388
383	DIPEP_WD	0.003262
295	DIPEP_QR	0.003234

COMPARAÇÃO MODELOS

	Modelo	ROC_AUC	F1	Precision	Recall	Balanced_Accuracy \
0	Random Forest	0.929265	0.833992	0.954751	0.740351	0.854550
1	XGBoost	0.945581	0.868275	0.921260	0.821053	0.879276
2	SVM	0.945362	0.885246	0.920455	0.852632	0.893503

MCC

0	0.735095
1	0.767207
2	0.792130

Fim avaliação / validação modelos

```
[9]: '''
      Funções para rodar a aplicação
      '''
      motifs = {
          "P_LOOP": r"[A-Z]G[A-Z]{4}GK[ST]",
          "KINASE2": r"[LIVM]{3,5}DD",
          "RNBS_A": r"FDL",
```

```

    "RNBS_B": r"G[RK]G",
    "RNBS_C": r"[A-Z]{2}P[A-Z]{2}W",
    "GLPL": r"GLPL",
    "RNBS_D": r"CFL",
    "MHD": r"MHD",
}

df_validacao = []

def validar_motifs(df,planta,modelo):
    print("\nVALIDAÇÃO BIOLÓGICA\n")
    counts = {}
    for motif_name, pattern in motifs.items():
        counts[motif_name] = 0
        for seq in df["protein_seq"]:
            if re.search(pattern, seq):
                counts[motif_name] += 1
    for k, v in counts.items():
        print(k, ":", v)

    counts['planta'] = planta
    counts['modelo'] = modelo

    df_validacao.append(counts)
    df_counts = pd.DataFrame([counts])
    df_counts.to_csv(
        f"{OUTPUT_DIR}/validacao_biologica_{modelo}_{planta}.csv",
        index=False
    )

def rodar_transdecoder(fasta_nt):
    cmd1 = [
        "TransDecoder.LongOrfs",
        "-t",
        fasta_nt,
        "--output_dir",
        f"{OUTPUT_DIR}"
    ]
    with open(fasta_nt+"_output1.txt", "a", encoding="utf-8") as log_file:
        subprocess.run(cmd1,
                        stdout=log_file,
                        stderr=subprocess.STDOUT,
                        text=True)

    cmd2 = [

```

```

        "TransDecoder.Predict",
        "-t",
        fasta_nt,
        "--output_dir",
        f"{OUTPUT_DIR}"
    ]
    with open(fasta_nt+"_output2.txt", "a", encoding="utf-8") as log_file:
        subprocess.run(cmd2,
                        stdout=log_file,
                        stderr=subprocess.STDOUT,
                        text=True)
    pep_file = fasta_nt + ".transdecoder.pep"
    return pep_file

def predizer_transcritos(fasta_nt,model,nmodel, output, planta):
    probabilidade = 0.85
    candidatos = []
    pep_file = rodar_transdecoder(fasta_nt)
    for record in SeqIO.parse(fasta_nt+".transdecoder.pep", "fasta"):
        protein = str(record.seq)
        feats = extrair_features_proteina(protein)
        if feats is None:
            continue
        X = scaler.transform([feats])
        prob = model.predict_proba(X)[0][1]
        candidatos.append({
            "id": record.id,
            "descricao": record.description,
            "protein_length": len(protein),
            "probabilidade_resistencia": prob,
            "protein_seq": protein
        })

    candidatos_df = pd.DataFrame(candidatos)
    candidatos_df = candidatos_df.sort_values(
        "probabilidade_resistencia",
        ascending=False
    )

    # top = candidatos_df.head(40)
    top = candidatos_df[candidatos_df["probabilidade_resistencia"] >=
    ↪probabilidade]
    print("\nTOP CANDIDATOS com probabilidade de >= {probabilidade}:
    ↪"+fasta_nt+", MODELO: "+nmodel)
    print(top[[
        "id",

```



```

        "descricao",
        "protein_length",
        "probabilidade_resistencia"
    ])

    validar_motifs(top,planta,nmodel)

    candidatos_df.to_csv(
        f"{OUTPUT_DIR}/{output}",
        index=False
    )

```

```

[10]: '''
Aplicação - Modelo XGBoost
'''

print("\nPredição para ARABIDOPSIS\n")
predizer_transcritos(transcriptoma_arabidopsis_fasta,
    ↳xgb_model, 'XGBoost', output="predicoes_arabidopsis_xgboost.
    ↳csv", planta="Arabidopsis thaliana")

print("\nPredição para MELONGENA\n")
predizer_transcritos(transcriptoma_melongena_fasta,
    ↳xgb_model, 'XGBoost', output="predicoes_melongena_xgboost.csv", planta="Solanum
    ↳melongena")

print("\nPredição para TUBEROSUM\n")
predizer_transcritos(transcriptoma_tuberosum_fasta,
    ↳xgb_model, 'XGBoost', output="predicoes_tuberosum_xgboost.csv", planta="Solanum
    ↳tuberosum")

print("\nPredição para PHYSALIS\n")
predizer_transcritos(transcriptoma_physalis_fasta,
    ↳xgb_model, 'XGBoost', output="predicoes_physalis_xgboost.csv", planta="Physalis
    ↳peruviana")

print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_xgboost.csv")

```

Predição para ARABIDOPSIS

TOP CANDIDATOS com probabilidade de >= {probabilidade}:

pipelineW/arabidopsis_transcriptoma.fasta, MODELO: XGBoost

	id	descricao \
644	AX507147.1.p1	AX507147.1.p1 GENE.AX507147.1~~AX507147.1.p1 ...

2694	PV383300.1.p1	PV383300.1.p1	GENE.PV383300.1~~PV383300.1.p1	...
2693	PV383299.1.p1	PV383299.1.p1	GENE.PV383299.1~~PV383299.1.p1	...
403	AX506335.1.p1	AX506335.1.p1	GENE.AX506335.1~~AX506335.1.p1	...
2633	LQ605013.1.p1	LQ605013.1.p1	GENE.LQ605013.1~~LQ605013.1.p1	...
1931	JB349452.1.p1	JB349452.1.p1	GENE.JB349452.1~~JB349452.1.p1	...
2572	LQ603265.1.p1	LQ603265.1.p1	GENE.LQ603265.1~~LQ603265.1.p1	...
2607	LQ603544.1.p1	LQ603544.1.p1	GENE.LQ603544.1~~LQ603544.1.p1	...
1870	JB347704.1.p1	JB347704.1.p1	GENE.JB347704.1~~JB347704.1.p1	...
1905	JB347983.1.p1	JB347983.1.p1	GENE.JB347983.1~~JB347983.1.p1	...
621	AX507060.1.p1	AX507060.1.p1	GENE.AX507060.1~~AX507060.1.p1	...
2598	LQ603450.1.p1	LQ603450.1.p1	GENE.LQ603450.1~~LQ603450.1.p1	...
1896	JB347889.1.p1	JB347889.1.p1	GENE.JB347889.1~~JB347889.1.p1	...
471	AX506539.1.p1	AX506539.1.p1	GENE.AX506539.1~~AX506539.1.p1	...
2588	LQ603401.1.p1	LQ603401.1.p1	GENE.LQ603401.1~~LQ603401.1.p1	...
1886	JB347840.1.p1	JB347840.1.p1	GENE.JB347840.1~~JB347840.1.p1	...
1857	JB347605.1.p1	JB347605.1.p1	GENE.JB347605.1~~JB347605.1.p1	...
2559	LQ603166.1.p1	LQ603166.1.p1	GENE.LQ603166.1~~LQ603166.1.p1	...
357	AX506167.1.p1	AX506167.1.p1	GENE.AX506167.1~~AX506167.1.p1	...
369	AX506207.1.p1	AX506207.1.p1	GENE.AX506207.1~~AX506207.1.p1	...
1435	FJ613384.1.p1	FJ613384.1.p1	GENE.FJ613384.1~~FJ613384.1.p1	...
2687	OP991902.1.p1	OP991902.1.p1	GENE.OP991902.1~~OP991902.1.p1	...
2371	JC297933.1.p1	JC297933.1.p1	GENE.JC297933.1~~JC297933.1.p1	...
2346	JC297678.1.p1	JC297678.1.p1	GENE.JC297678.1~~JC297678.1.p1	...
2501	LC214893.1.p1	LC214893.1.p1	GENE.LC214893.1~~LC214893.1.p1	...
668	AX507220.1.p1	AX507220.1.p1	GENE.AX507220.1~~AX507220.1.p1	...
2688	OP991902.1.p2	OP991902.1.p2	GENE.OP991902.1~~OP991902.1.p2	...
2485	LC214889.1.p2	LC214889.1.p2	GENE.LC214889.1~~LC214889.1.p2	...
410	AX506365.1.p1	AX506365.1.p1	GENE.AX506365.1~~AX506365.1.p1	...
2474	LC214887.1.p2	LC214887.1.p2	GENE.LC214887.1~~LC214887.1.p2	...
2480	LC214888.1.p1	LC214888.1.p1	GENE.LC214888.1~~LC214888.1.p1	...
2489	LC214890.1.p2	LC214890.1.p2	GENE.LC214890.1~~LC214890.1.p2	...
2491	LC214891.1.p2	LC214891.1.p2	GENE.LC214891.1~~LC214891.1.p2	...
2505	LC214894.1.p2	LC214894.1.p2	GENE.LC214894.1~~LC214894.1.p2	...
2017	JC208765.1.p1	JC208765.1.p1	GENE.JC208765.1~~JC208765.1.p1	...
2365	JC297794.1.p1	JC297794.1.p1	GENE.JC297794.1~~JC297794.1.p1	...
2018	JC208766.1.p1	JC208766.1.p1	GENE.JC208766.1~~JC208766.1.p1	...
2005	JC208732.1.p1	JC208732.1.p1	GENE.JC208732.1~~JC208732.1.p1	...
2217	JC209125.1.p1	JC209125.1.p1	GENE.JC209125.1~~JC209125.1.p1	...
1436	FJ773993.1.p1	FJ773993.1.p1	GENE.FJ773993.1~~FJ773993.1.p1	...
2730	U91995.1.p1	U91995.1.p1	GENE.U91995.1~~U91995.1.p1	ORF ty...
2306	JC209269.1.p1	JC209269.1.p1	GENE.JC209269.1~~JC209269.1.p1	...
1015	AY016208.1.p1	AY016208.1.p1	GENE.AY016208.1~~AY016208.1.p1	...

	protein_length	probabilidade_resistencia
644	894	0.997513
2694	777	0.997141
2693	1165	0.996910
403	1405	0.995276

2633	926	0.995031
1931	926	0.995031
2572	900	0.994000
2607	900	0.994000
1870	900	0.994000
1905	900	0.994000
621	853	0.993828
2598	1204	0.993551
1896	1204	0.993551
471	1257	0.993024
2588	890	0.992967
1886	890	0.992967
1857	1189	0.992680
2559	1189	0.992680
357	1171	0.989623
369	1218	0.987729
1435	1164	0.986737
2687	562	0.983845
2371	1164	0.980818
2346	1165	0.978316
2501	575	0.976378
668	1168	0.972475
2688	306	0.968801
2485	283	0.965638
410	1215	0.964493
2474	384	0.964308
2480	572	0.964282
2489	384	0.960036
2491	410	0.956241
2505	410	0.955513
2017	457	0.954748
2365	969	0.954010
2018	201	0.952426
2005	200	0.937600
2217	107	0.891729
1436	602	0.890404
2730	1049	0.881138
2306	186	0.869569
1015	1644	0.851698

VALIDAÇÃO BIOLÓGICA

P_LOOP : 34
 KINASE2 : 31
 RNBS_A : 13
 RNBS_B : 5
 RNBS_C : 8
 GLPL : 15

RNBS_D : 18
MHD : 18

Predição para MELONGENA

TOP CANDIDATOS com probabilidade de >= {probabilidade}:
pipelineW/melongena_transcriptoma.fasta, MODELO: XGBoost

	id		descricao \
267	GBEF01001042.1.p1	GBEF01001042.1.p1	GENE.GBEF01001042.1~~GBEF010...
9150	GBEF01044247.1.p1	GBEF01044247.1.p1	GENE.GBEF01044247.1~~GBEF010...
4481	GBEF01026076.1.p1	GBEF01026076.1.p1	GENE.GBEF01026076.1~~GBEF010...
9154	GBEF01044251.1.p1	GBEF01044251.1.p1	GENE.GBEF01044251.1~~GBEF010...
9152	GBEF01044249.1.p1	GBEF01044249.1.p1	GENE.GBEF01044249.1~~GBEF010...
...	...	...	...
8080	GBEF01041828.1.p2	GBEF01041828.1.p2	GENE.GBEF01041828.1~~GBEF010...
8068	GBEF01041821.1.p3	GBEF01041821.1.p3	GENE.GBEF01041821.1~~GBEF010...
8078	GBEF01041827.1.p2	GBEF01041827.1.p2	GENE.GBEF01041827.1~~GBEF010...
9228	GBEF01044457.1.p2	GBEF01044457.1.p2	GENE.GBEF01044457.1~~GBEF010...
9231	GBEF01044459.1.p1	GBEF01044459.1.p1	GENE.GBEF01044459.1~~GBEF010...
	protein_length	probabilidade_resistencia	
267	867	0.996292	
9150	713	0.993575	
4481	892	0.991818	
9154	532	0.991787	
9152	579	0.991738	
...	...	...	
8080	133	0.855075	
8068	133	0.855075	
8078	133	0.855075	
9228	244	0.852566	
9231	244	0.852566	

[68 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 14
KINASE2 : 33
RNBS_A : 13
RNBS_B : 25
RNBS_C : 18
GLPL : 16
RNBS_D : 16
MHD : 2

Predição para TUBEROSUM

TOP CANDIDATOS com probabilidade de >= {probabilidade}:
 pipelineW/tuberosum_transcriptoma.fasta, MODELO: XGBoost

	id	descricao \
45	EF137705.1.p1	EF137705.1.p1 GENE.EF137705.1~~EF137705.1.p1 ...
43	BD391474.1.p1	BD391474.1.p1 GENE.BD391474.1~~BD391474.1.p1 ...
268	PP061819.1.p1	PP061819.1.p1 GENE.PP061819.1~~PP061819.1.p1 ...
21	AJ300266.1.p1	AJ300266.1.p1 GENE.AJ300266.1~~AJ300266.1.p1 ...
273	PP061822.1.p2	PP061822.1.p2 GENE.PP061822.1~~PP061822.1.p2 ...
259	PP061811.1.p2	PP061811.1.p2 GENE.PP061811.1~~PP061811.1.p2 ...
265	PP061818.1.p2	PP061818.1.p2 GENE.PP061818.1~~PP061818.1.p2 ...
255	PP061808.1.p2	PP061808.1.p2 GENE.PP061808.1~~PP061808.1.p2 ...
250	PP061795.1.p2	PP061795.1.p2 GENE.PP061795.1~~PP061795.1.p2 ...
22	AJ300266.1.p2	AJ300266.1.p2 GENE.AJ300266.1~~AJ300266.1.p2 ...
31	AJ507316.1.p1	AJ507316.1.p1 GENE.AJ507316.1~~AJ507316.1.p1 ...
266	PP061819.1.p4	PP061819.1.p4 GENE.PP061819.1~~PP061819.1.p4 ...
258	PP061811.1.p1	PP061811.1.p1 GENE.PP061811.1~~PP061811.1.p1 ...
319	Z11741.1.p1	Z11741.1.p1 GENE.Z11741.1~~Z11741.1.p1 ORF ty...
251	PP061795.1.p1	PP061795.1.p1 GENE.PP061795.1~~PP061795.1.p1 ...

	protein_length	probabilidade_resistencia
45	849	0.995570
43	871	0.991622
268	337	0.967316
21	486	0.958976
273	337	0.954708
259	337	0.954208
265	337	0.951565
255	337	0.951565
250	337	0.951565
22	333	0.927360
31	509	0.887240
266	113	0.882317
258	471	0.880278
319	510	0.867329
251	478	0.861211

VALIDAÇÃO BIOLÓGICA

P_LOOP : 10
 KINASE2 : 3
 RNBS_A : 1
 RNBS_B : 5
 RNBS_C : 1
 GLPL : 8
 RNBS_D : 9
 MHD : 2

Predição para PHYSALIS

TOP CANDIDATOS com probabilidade de $\geq$ {probabilidade}:
 pipelineW/physalis_transcriptoma.fasta, MODELO: XGBoost

	id		descricao \
12682	J0138557.1.p1	J0138557.1.p1	GENE.J0138557.1~~J0138557.1.p1 ...
8851	J0132546.1.p1	J0132546.1.p1	GENE.J0132546.1~~J0132546.1.p1 ...
8856	J0132548.1.p1	J0132548.1.p1	GENE.J0132548.1~~J0132548.1.p1 ...
13163	J0138904.1.p1	J0138904.1.p1	GENE.J0138904.1~~J0138904.1.p1 ...
941	J0125565.1.p1	J0125565.1.p1	GENE.J0125565.1~~J0125565.1.p1 ...
...	...	...	...
8752	J0132049.1.p2	J0132049.1.p2	GENE.J0132049.1~~J0132049.1.p2 ...
8750	J0132048.1.p2	J0132048.1.p2	GENE.J0132048.1~~J0132048.1.p2 ...
11488	J0134644.1.p1	J0134644.1.p1	GENE.J0134644.1~~J0134644.1.p1 ...
12925	J0138727.1.p3	J0138727.1.p3	GENE.J0138727.1~~J0138727.1.p3 ...
13661	J0139282.1.p1	J0139282.1.p1	GENE.J0139282.1~~J0139282.1.p1 ...
	protein_length	probabilidade_resistencia	
12682	888	0.998277	
8851	776	0.997731	
8856	639	0.997535	
13163	884	0.997354	
941	646	0.997114	
...	...	...	
8752	165	0.852510	
8750	165	0.852510	
11488	181	0.852232	
12925	110	0.851489	
13661	735	0.850189	

[279 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 103
 KINASE2 : 132
 RNBS_A : 27
 RNBS_B : 46
 RNBS_C : 36
 GLPL : 60
 RNBS_D : 71
 MHD : 24

Arquivo salvo:
 pipelineW/predicoes_xgboost.csv

```
[11]: print(df_validacao)
```

```
[{'P_LOOP': 34, 'KINASE2': 31, 'RNBS_A': 13, 'RNBS_B': 5, 'RNBS_C': 8, 'GLPL': 15, 'RNBS_D': 18, 'MHD': 18, 'planta': 'Arabidopsis thaliana', 'modelo': 'XGBoost'}, {'P_LOOP': 14, 'KINASE2': 33, 'RNBS_A': 13, 'RNBS_B': 25, 'RNBS_C': 18, 'GLPL': 16, 'RNBS_D': 16, 'MHD': 2, 'planta': 'Solanum melongena', 'modelo': 'XGBoost'}, {'P_LOOP': 10, 'KINASE2': 3, 'RNBS_A': 1, 'RNBS_B': 5, 'RNBS_C': 1, 'GLPL': 8, 'RNBS_D': 9, 'MHD': 2, 'planta': 'Solanum tuberosum', 'modelo': 'XGBoost'}, {'P_LOOP': 103, 'KINASE2': 132, 'RNBS_A': 27, 'RNBS_B': 46, 'RNBS_C': 36, 'GLPL': 60, 'RNBS_D': 71, 'MHD': 24, 'planta': 'Physalis peruviana', 'modelo': 'XGBoost'}]
```

```
[12]: '''
Aplicação - Modelo Random Forest
'''

print("\nPredição para ARABIDOPSIS\n")
predizer_transcritos(transcriptoma_arabidopsis_fasta,
    rf_model, "RF", output="predicoes_arabidopsis_rf.csv", planta="Arabidopsis_
    thaliana")

print("\nPredição para MELONGENA\n")
predizer_transcritos(transcriptoma_melongena_fasta,
    rf_model, "RF", output="predicoes_melongena_rf.csv", planta="Solanum melongena")

print("\nPredição para TUBEROSUM\n")
predizer_transcritos(transcriptoma_tuberosum_fasta,
    rf_model, "RF", output="predicoes_tuberosum_rf.csv", planta="Solanum tuberosum")

print("\nPredição para PHYSALIS\n")
predizer_transcritos(transcriptoma_physalis_fasta,
    rf_model, "RF", output="predicoes_physalis_rf.csv", planta="Physalis peruviana")

print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_rf.csv")
```

Predição para ARABIDOPSIS

TOP CANDIDATOS com probabilidade de $\geq$ {probabilidade}:

pipelineW/arabidopsis_transcriptoma.fasta, MODELO: RF

	id	descricao \
2633	LQ605013.1.p1	LQ605013.1.p1 GENE.LQ605013.1~~LQ605013.1.p1 ...
1931	JB349452.1.p1	JB349452.1.p1 GENE.JB349452.1~~JB349452.1.p1 ...
1857	JB347605.1.p1	JB347605.1.p1 GENE.JB347605.1~~JB347605.1.p1 ...
2559	LQ603166.1.p1	LQ603166.1.p1 GENE.LQ603166.1~~LQ603166.1.p1 ...
1896	JB347889.1.p1	JB347889.1.p1 GENE.JB347889.1~~JB347889.1.p1 ...
2598	LQ603450.1.p1	LQ603450.1.p1 GENE.LQ603450.1~~LQ603450.1.p1 ...

1870	JB347704.1.p1	JB347704.1.p1	GENE.JB347704.1~~JB347704.1.p1	...
1905	JB347983.1.p1	JB347983.1.p1	GENE.JB347983.1~~JB347983.1.p1	...
2607	LQ603544.1.p1	LQ603544.1.p1	GENE.LQ603544.1~~LQ603544.1.p1	...
2572	LQ603265.1.p1	LQ603265.1.p1	GENE.LQ603265.1~~LQ603265.1.p1	...
1886	JB347840.1.p1	JB347840.1.p1	GENE.JB347840.1~~JB347840.1.p1	...
2588	LQ603401.1.p1	LQ603401.1.p1	GENE.LQ603401.1~~LQ603401.1.p1	...
2693	PV383299.1.p1	PV383299.1.p1	GENE.PV383299.1~~PV383299.1.p1	...
471	AX506539.1.p1	AX506539.1.p1	GENE.AX506539.1~~AX506539.1.p1	...
621	AX507060.1.p1	AX507060.1.p1	GENE.AX507060.1~~AX507060.1.p1	...
403	AX506335.1.p1	AX506335.1.p1	GENE.AX506335.1~~AX506335.1.p1	...
2694	PV383300.1.p1	PV383300.1.p1	GENE.PV383300.1~~PV383300.1.p1	...
644	AX507147.1.p1	AX507147.1.p1	GENE.AX507147.1~~AX507147.1.p1	...
369	AX506207.1.p1	AX506207.1.p1	GENE.AX506207.1~~AX506207.1.p1	...
357	AX506167.1.p1	AX506167.1.p1	GENE.AX506167.1~~AX506167.1.p1	...
1435	FJ613384.1.p1	FJ613384.1.p1	GENE.FJ613384.1~~FJ613384.1.p1	...
2371	JC297933.1.p1	JC297933.1.p1	GENE.JC297933.1~~JC297933.1.p1	...
2346	JC297678.1.p1	JC297678.1.p1	GENE.JC297678.1~~JC297678.1.p1	...
2501	LC214893.1.p1	LC214893.1.p1	GENE.LC214893.1~~LC214893.1.p1	...

	protein_length	probabilidade_resistencia
2633	926	0.985971
1931	926	0.985971
1857	1189	0.982410
2559	1189	0.982410
1896	1204	0.981637
2598	1204	0.981637
1870	900	0.976018
1905	900	0.976018
2607	900	0.976018
2572	900	0.976018
1886	890	0.974516
2588	890	0.974516
2693	1165	0.967829
471	1257	0.957154
621	853	0.947539
403	1405	0.946782
2694	777	0.946612
644	894	0.919111
369	1218	0.916670
357	1171	0.894594
1435	1164	0.888327
2371	1164	0.887923
2346	1165	0.874062
2501	575	0.864262

VALIDAÇÃO BIOLÓGICA

P_LOOP : 24

KINASE2 : 19
 RNBS_A : 11
 RNBS_B : 1
 RNBS_C : 6
 GLPL : 14
 RNBS_D : 9
 MHD : 16

Predição para MELONGENA

TOP CANDIDATOS com probabilidade de >= {probabilidade}:
 pipelineW/melongena_transcriptoma.fasta, MODELO: RF

	id	descricao \
267	GBEF01001042.1.p1	GBEF01001042.1.p1 GENE.GBEF01001042.1~~GBEF010...
4481	GBEF01026076.1.p1	GBEF01026076.1.p1 GENE.GBEF01026076.1~~GBEF010...
4482	GBEF01026077.1.p1	GBEF01026077.1.p1 GENE.GBEF01026077.1~~GBEF010...
4331	GBEF01025818.1.p1	GBEF01025818.1.p1 GENE.GBEF01025818.1~~GBEF010...
4330	GBEF01025817.1.p1	GBEF01025817.1.p1 GENE.GBEF01025817.1~~GBEF010...
3214	GBEF01023314.1.p1	GBEF01023314.1.p1 GENE.GBEF01023314.1~~GBEF010...
3213	GBEF01023313.1.p1	GBEF01023313.1.p1 GENE.GBEF01023313.1~~GBEF010...
9150	GBEF01044247.1.p1	GBEF01044247.1.p1 GENE.GBEF01044247.1~~GBEF010...
3215	GBEF01023315.1.p1	GBEF01023315.1.p1 GENE.GBEF01023315.1~~GBEF010...
9151	GBEF01044248.1.p1	GBEF01044248.1.p1 GENE.GBEF01044248.1~~GBEF010...
3216	GBEF01023316.1.p1	GBEF01023316.1.p1 GENE.GBEF01023316.1~~GBEF010...
9152	GBEF01044249.1.p1	GBEF01044249.1.p1 GENE.GBEF01044249.1~~GBEF010...
9153	GBEF01044250.1.p1	GBEF01044250.1.p1 GENE.GBEF01044250.1~~GBEF010...
9154	GBEF01044251.1.p1	GBEF01044251.1.p1 GENE.GBEF01044251.1~~GBEF010...
9155	GBEF01044252.1.p1	GBEF01044252.1.p1 GENE.GBEF01044252.1~~GBEF010...

	protein_length	probabilidade_resistencia
267	867	0.972132
4481	892	0.968710
4482	866	0.960375
4331	886	0.954867
4330	886	0.954867
3214	872	0.946853
3213	881	0.946199
9150	713	0.919562
3215	883	0.910912
9151	706	0.908241
3216	874	0.898575
9152	579	0.889363
9153	572	0.888625
9154	532	0.879422
9155	525	0.878795

VALIDAÇÃO BIOLÓGICA

P_LOOP : 9
 KINASE2 : 15
 RNBS_A : 4
 RNBS_B : 7
 RNBS_C : 0
 GLPL : 11
 RNBS_D : 13
 MHD : 2

Predição para TUBEROSUM

TOP CANDIDATOS com probabilidade de >= {probabilidade}:
 pipelineW/tuberosum_transcriptoma.fasta, MODELO: RF

	id	descricao \
43	BD391474.1.p1	BD391474.1.p1 GENE.BD391474.1~~BD391474.1.p1 ...
45	EF137705.1.p1	EF137705.1.p1 GENE.EF137705.1~~EF137705.1.p1 ...

	protein_length	probabilidade_resistencia
43	871	0.979041
45	849	0.960364

VALIDAÇÃO BIOLÓGICA

P_LOOP : 2
 KINASE2 : 2
 RNBS_A : 1
 RNBS_B : 0
 RNBS_C : 0
 GLPL : 1
 RNBS_D : 2
 MHD : 1

Predição para PHYSALIS

TOP CANDIDATOS com probabilidade de >= {probabilidade}:
 pipelineW/physalis_transcriptoma.fasta, MODELO: RF

	id	descricao \
12825	J0138656.1.p1	J0138656.1.p1 GENE.J0138656.1~~J0138656.1.p1 ...
8377	J0131689.1.p1	J0131689.1.p1 GENE.J0131689.1~~J0131689.1.p1 ...
12682	J0138557.1.p1	J0138557.1.p1 GENE.J0138557.1~~J0138557.1.p1 ...
12868	J0138683.1.p1	J0138683.1.p1 GENE.J0138683.1~~J0138683.1.p1 ...
8378	J0131690.1.p1	J0131690.1.p1 GENE.J0131690.1~~J0131690.1.p1 ...
13163	J0138904.1.p1	J0138904.1.p1 GENE.J0138904.1~~J0138904.1.p1 ...
10868	J0134210.1.p1	J0134210.1.p1 GENE.J0134210.1~~J0134210.1.p1 ...
10869	J0134211.1.p1	J0134211.1.p1 GENE.J0134211.1~~J0134211.1.p1 ...

12659	J0138542.1.p1	J0138542.1.p1	GENE.J0138542.1~~J0138542.1.p1	...
10806	J0134174.1.p1	J0134174.1.p1	GENE.J0134174.1~~J0134174.1.p1	...
10807	J0134175.1.p1	J0134175.1.p1	GENE.J0134175.1~~J0134175.1.p1	...
9439	J0133073.1.p1	J0133073.1.p1	GENE.J0133073.1~~J0133073.1.p1	...
9436	J0133072.1.p1	J0133072.1.p1	GENE.J0133072.1~~J0133072.1.p1	...
8986	J0132682.1.p1	J0132682.1.p1	GENE.J0132682.1~~J0132682.1.p1	...
9657	J0133194.1.p1	J0133194.1.p1	GENE.J0133194.1~~J0133194.1.p1	...
12877	J0138694.1.p1	J0138694.1.p1	GENE.J0138694.1~~J0138694.1.p1	...
8851	J0132546.1.p1	J0132546.1.p1	GENE.J0132546.1~~J0132546.1.p1	...
13273	J0138990.1.p1	J0138990.1.p1	GENE.J0138990.1~~J0138990.1.p1	...
8987	J0132683.1.p1	J0132683.1.p1	GENE.J0132683.1~~J0132683.1.p1	...
8853	J0132547.1.p1	J0132547.1.p1	GENE.J0132547.1~~J0132547.1.p1	...
13342	J0139035.1.p1	J0139035.1.p1	GENE.J0139035.1~~J0139035.1.p1	...
8856	J0132548.1.p1	J0132548.1.p1	GENE.J0132548.1~~J0132548.1.p1	...
7434	J0130641.1.p1	J0130641.1.p1	GENE.J0130641.1~~J0130641.1.p1	...
13130	J0138882.1.p1	J0138882.1.p1	GENE.J0138882.1~~J0138882.1.p1	...
13937	J0139498.1.p1	J0139498.1.p1	GENE.J0139498.1~~J0139498.1.p1	...
8858	J0132549.1.p1	J0132549.1.p1	GENE.J0132549.1~~J0132549.1.p1	...
9745	J0133251.1.p1	J0133251.1.p1	GENE.J0133251.1~~J0133251.1.p1	...
9744	J0133250.1.p1	J0133250.1.p1	GENE.J0133250.1~~J0133250.1.p1	...
7432	J0130640.1.p1	J0130640.1.p1	GENE.J0130640.1~~J0130640.1.p1	...
941	J0125565.1.p1	J0125565.1.p1	GENE.J0125565.1~~J0125565.1.p1	...
9694	J0133214.1.p1	J0133214.1.p1	GENE.J0133214.1~~J0133214.1.p1	...
9695	J0133215.1.p1	J0133215.1.p1	GENE.J0133215.1~~J0133215.1.p1	...
10776	J0134148.1.p1	J0134148.1.p1	GENE.J0134148.1~~J0134148.1.p1	...
14048	J0139589.1.p1	J0139589.1.p1	GENE.J0139589.1~~J0139589.1.p1	...
11586	J0134734.1.p1	J0134734.1.p1	GENE.J0134734.1~~J0134734.1.p1	...
940	J0125564.1.p1	J0125564.1.p1	GENE.J0125564.1~~J0125564.1.p1	...
6727	J0130096.1.p1	J0130096.1.p1	GENE.J0130096.1~~J0130096.1.p1	...
4028	J0127791.1.p1	J0127791.1.p1	GENE.J0127791.1~~J0127791.1.p1	...
4022	J0127789.1.p1	J0127789.1.p1	GENE.J0127789.1~~J0127789.1.p1	...
4025	J0127790.1.p1	J0127790.1.p1	GENE.J0127790.1~~J0127790.1.p1	...
4019	J0127788.1.p1	J0127788.1.p1	GENE.J0127788.1~~J0127788.1.p1	...
9646	J0133187.1.p1	J0133187.1.p1	GENE.J0133187.1~~J0133187.1.p1	...
14553	J0140036.1.p1	J0140036.1.p1	GENE.J0140036.1~~J0140036.1.p1	...
6723	J0130094.1.p1	J0130094.1.p1	GENE.J0130094.1~~J0130094.1.p1	...
4033	J0127792.1.p1	J0127792.1.p1	GENE.J0127792.1~~J0127792.1.p1	...
4041	J0127795.1.p1	J0127795.1.p1	GENE.J0127795.1~~J0127795.1.p1	...
4039	J0127794.1.p1	J0127794.1.p1	GENE.J0127794.1~~J0127794.1.p1	...
4035	J0127793.1.p1	J0127793.1.p1	GENE.J0127793.1~~J0127793.1.p1	...
12406	J0138325.1.p1	J0138325.1.p1	GENE.J0138325.1~~J0138325.1.p1	...
6049	J0129492.1.p1	J0129492.1.p1	GENE.J0129492.1~~J0129492.1.p1	...
6051	J0129493.1.p1	J0129493.1.p1	GENE.J0129493.1~~J0129493.1.p1	...
6048	J0129491.1.p1	J0129491.1.p1	GENE.J0129491.1~~J0129491.1.p1	...

	protein_length	probabilidade_resistencia
12825	1150	0.987320
8377	968	0.986820

12682	888	0.986129
12868	1148	0.983354
8378	920	0.981130
13163	884	0.980646
10868	865	0.977279
10869	865	0.977279
12659	1068	0.965681
10806	892	0.965531
10807	892	0.965531
9439	938	0.962926
9436	938	0.962926
8986	889	0.961035
9657	932	0.960331
12877	806	0.959111
8851	776	0.957836
13273	885	0.957497
8987	834	0.956030
8853	759	0.953272
13342	735	0.951491
8856	639	0.949170
7434	673	0.947578
13130	841	0.945749
13937	669	0.945672
8858	622	0.943277
9745	618	0.938890
9744	618	0.935228
7432	566	0.914502
941	646	0.910050
9694	683	0.907383
9695	551	0.898312
10776	1021	0.897311
14048	476	0.895883
11586	723	0.887630
940	435	0.875295
6727	615	0.872811
4028	697	0.872769
4022	697	0.872769
4025	697	0.872769
4019	697	0.872769
9646	670	0.871096
14553	370	0.869055
6723	600	0.864597
4033	500	0.858376
4041	500	0.858376
4039	500	0.858376
4035	500	0.858376
12406	868	0.857533
6049	387	0.853910

6051	387	0.853910
6048	387	0.853910

VALIDAÇÃO BIOLÓGICA

P_LOOP : 46
 KINASE2 : 50
 RNBS_A : 6
 RNBS_B : 15
 RNBS_C : 8
 GLPL : 47
 RNBS_D : 35
 MHD : 18

Arquivo salvo:
 pipelineW/predicoes_rf.csv

```

[13]: '''
      Aplicação - Modelo SVM
      '''

      print("\nPredição para ARABIDOPSIS\n")
      predizer_transcritos(transcriptoma_arabidopsis_fasta,
        ↪svm_model, "SVM", output="predicoes_arabidopsis_svm.csv", planta="Arabidopsis_
        ↪thaliana")

      print("\nPredição para MELONGENA\n")
      predizer_transcritos(transcriptoma_melongena_fasta,
        ↪svm_model, 'SVM', output="predicoes_melongena_svm.csv", planta="Solanum_
        ↪melongena")

      print("\nPredição para TUBEROSUM\n")
      predizer_transcritos(transcriptoma_tuberosum_fasta,
        ↪svm_model, "SVM", output="predicoes_tuberosum_svm.csv", planta="Solanum_
        ↪tuberosum")

      print("\nPredição para PHYSALIS\n")
      predizer_transcritos(transcriptoma_physalis_fasta,
        ↪svm_model, "SVM", output="predicoes_physalis_svm.csv", planta="Physalis_
        ↪peruviana")

      print("\nArquivo salvo:")
      print(f"{OUTPUT_DIR}/predicoes_svm.csv")
  
```

Predição para ARABIDOPSIS

TOP CANDIDATOS com probabilidade de $\geq$ {probabilidade}:
 pipelineW/arabidopsis_transcriptoma.fasta, MODELO: SVM

	id		descricao \
2598	LQ603450.1.p1	LQ603450.1.p1	GENE.LQ603450.1~~LQ603450.1.p1 ...
1896	JB347889.1.p1	JB347889.1.p1	GENE.JB347889.1~~JB347889.1.p1 ...
1857	JB347605.1.p1	JB347605.1.p1	GENE.JB347605.1~~JB347605.1.p1 ...
2559	LQ603166.1.p1	LQ603166.1.p1	GENE.LQ603166.1~~LQ603166.1.p1 ...
2694	PV383300.1.p1	PV383300.1.p1	GENE.PV383300.1~~PV383300.1.p1 ...
471	AX506539.1.p1	AX506539.1.p1	GENE.AX506539.1~~AX506539.1.p1 ...
1905	JB347983.1.p1	JB347983.1.p1	GENE.JB347983.1~~JB347983.1.p1 ...
1870	JB347704.1.p1	JB347704.1.p1	GENE.JB347704.1~~JB347704.1.p1 ...
2607	LQ603544.1.p1	LQ603544.1.p1	GENE.LQ603544.1~~LQ603544.1.p1 ...
2572	LQ603265.1.p1	LQ603265.1.p1	GENE.LQ603265.1~~LQ603265.1.p1 ...
2588	LQ603401.1.p1	LQ603401.1.p1	GENE.LQ603401.1~~LQ603401.1.p1 ...
1886	JB347840.1.p1	JB347840.1.p1	GENE.JB347840.1~~JB347840.1.p1 ...
644	AX507147.1.p1	AX507147.1.p1	GENE.AX507147.1~~AX507147.1.p1 ...
2693	PV383299.1.p1	PV383299.1.p1	GENE.PV383299.1~~PV383299.1.p1 ...
2633	LQ605013.1.p1	LQ605013.1.p1	GENE.LQ605013.1~~LQ605013.1.p1 ...
1931	JB349452.1.p1	JB349452.1.p1	GENE.JB349452.1~~JB349452.1.p1 ...
410	AX506365.1.p1	AX506365.1.p1	GENE.AX506365.1~~AX506365.1.p1 ...
621	AX507060.1.p1	AX507060.1.p1	GENE.AX507060.1~~AX507060.1.p1 ...
403	AX506335.1.p1	AX506335.1.p1	GENE.AX506335.1~~AX506335.1.p1 ...
2501	LC214893.1.p1	LC214893.1.p1	GENE.LC214893.1~~LC214893.1.p1 ...
2688	OP991902.1.p2	OP991902.1.p2	GENE.OP991902.1~~OP991902.1.p2 ...
2480	LC214888.1.p1	LC214888.1.p1	GENE.LC214888.1~~LC214888.1.p1 ...
369	AX506207.1.p1	AX506207.1.p1	GENE.AX506207.1~~AX506207.1.p1 ...
2687	OP991902.1.p1	OP991902.1.p1	GENE.OP991902.1~~OP991902.1.p1 ...
2017	JC208765.1.p1	JC208765.1.p1	GENE.JC208765.1~~JC208765.1.p1 ...
2505	LC214894.1.p2	LC214894.1.p2	GENE.LC214894.1~~LC214894.1.p2 ...
2491	LC214891.1.p2	LC214891.1.p2	GENE.LC214891.1~~LC214891.1.p2 ...
2730	U91995.1.p1	U91995.1.p1	GENE.U91995.1~~U91995.1.p1 ORF ty...
2485	LC214889.1.p2	LC214889.1.p2	GENE.LC214889.1~~LC214889.1.p2 ...
2371	JC297933.1.p1	JC297933.1.p1	GENE.JC297933.1~~JC297933.1.p1 ...
233	AX505699.1.p1	AX505699.1.p1	GENE.AX505699.1~~AX505699.1.p1 ...
357	AX506167.1.p1	AX506167.1.p1	GENE.AX506167.1~~AX506167.1.p1 ...
1435	FJ613384.1.p1	FJ613384.1.p1	GENE.FJ613384.1~~FJ613384.1.p1 ...
2474	LC214887.1.p2	LC214887.1.p2	GENE.LC214887.1~~LC214887.1.p2 ...
2489	LC214890.1.p2	LC214890.1.p2	GENE.LC214890.1~~LC214890.1.p2 ...
2346	JC297678.1.p1	JC297678.1.p1	GENE.JC297678.1~~JC297678.1.p1 ...
2365	JC297794.1.p1	JC297794.1.p1	GENE.JC297794.1~~JC297794.1.p1 ...
705	AX507341.1.p1	AX507341.1.p1	GENE.AX507341.1~~AX507341.1.p1 ...
479	AX506573.1.p1	AX506573.1.p1	GENE.AX506573.1~~AX506573.1.p1 ...

	protein_length	probabilidade_resistencia
2598	1204	0.999994
1896	1204	0.999994
1857	1189	0.999994
2559	1189	0.999994

2694	777	0.999989
471	1257	0.999987
1905	900	0.996735
1870	900	0.996735
2607	900	0.996735
2572	900	0.996735
2588	890	0.996110
1886	890	0.996110
644	894	0.995184
2693	1165	0.994133
2633	926	0.993297
1931	926	0.993297
410	1215	0.993057
621	853	0.991835
403	1405	0.987941
2501	575	0.985270
2688	306	0.977584
2480	572	0.974933
369	1218	0.971891
2687	562	0.961882
2017	457	0.961834
2505	410	0.953254
2491	410	0.951696
2730	1049	0.948885
2485	283	0.947430
2371	1164	0.940983
233	931	0.940714
357	1171	0.937207
1435	1164	0.936180
2474	384	0.933645
2489	384	0.918570
2346	1165	0.918535
2365	969	0.903959
705	836	0.895389
479	906	0.859467

VALIDAÇÃO BIOLÓGICA

P_LOOP : 33
 KINASE2 : 27
 RNBS_A : 13
 RNBS_B : 3
 RNBS_C : 8
 GLPL : 16
 RNBS_D : 16
 MHD : 17

Predição para MELONGENA

TOP CANDIDATOS com probabilidade de >= {probabilidade}:

pipelineW/melongena_transcriptoma.fasta, MODELO: SVM

	id	descricao \
9150	GBEF01044247.1.p1	GBEF01044247.1.p1 GENE.GBEF01044247.1~~GBEF010...
267	GBEF01001042.1.p1	GBEF01001042.1.p1 GENE.GBEF01001042.1~~GBEF010...
9152	GBEF01044249.1.p1	GBEF01044249.1.p1 GENE.GBEF01044249.1~~GBEF010...
4481	GBEF01026076.1.p1	GBEF01026076.1.p1 GENE.GBEF01026076.1~~GBEF010...
9154	GBEF01044251.1.p1	GBEF01044251.1.p1 GENE.GBEF01044251.1~~GBEF010...
...	...	...
1911	GBEF01016846.1.p1	GBEF01016846.1.p1 GENE.GBEF01016846.1~~GBEF010...
8488	GBEF01042733.1.p1	GBEF01042733.1.p1 GENE.GBEF01042733.1~~GBEF010...
8489	GBEF01042734.1.p1	GBEF01042734.1.p1 GENE.GBEF01042734.1~~GBEF010...
8221	GBEF01042072.1.p1	GBEF01042072.1.p1 GENE.GBEF01042072.1~~GBEF010...
7480	GBEF01035527.1.p2	GBEF01035527.1.p2 GENE.GBEF01035527.1~~GBEF010...

	protein_length	probabilidade_resistencia
9150	713	0.999998
267	867	0.999997
9152	579	0.999996
4481	892	0.999991
9154	532	0.999990
...	...	...
1911	698	0.865384
8488	394	0.863457
8489	394	0.863457
8221	608	0.860141
7480	409	0.850817

[69 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 11
 KINASE2 : 23
 RNBS_A : 11
 RNBS_B : 20
 RNBS_C : 16
 GLPL : 13
 RNBS_D : 22
 MHD : 4

Predição para TUBEROSUM

TOP CANDIDATOS com probabilidade de >= {probabilidade}:

pipelineW/tuberosum_transcriptoma.fasta, MODELO: SVM

	id	descricao \
45	EF137705.1.p1	EF137705.1.p1 GENE.EF137705.1~~EF137705.1.p1 ...
21	AJ300266.1.p1	AJ300266.1.p1 GENE.AJ300266.1~~AJ300266.1.p1 ...
259	PP061811.1.p2	PP061811.1.p2 GENE.PP061811.1~~PP061811.1.p2 ...
265	PP061818.1.p2	PP061818.1.p2 GENE.PP061818.1~~PP061818.1.p2 ...
250	PP061795.1.p2	PP061795.1.p2 GENE.PP061795.1~~PP061795.1.p2 ...
255	PP061808.1.p2	PP061808.1.p2 GENE.PP061808.1~~PP061808.1.p2 ...
273	PP061822.1.p2	PP061822.1.p2 GENE.PP061822.1~~PP061822.1.p2 ...
258	PP061811.1.p1	PP061811.1.p1 GENE.PP061811.1~~PP061811.1.p1 ...
272	PP061822.1.p1	PP061822.1.p1 GENE.PP061822.1~~PP061822.1.p1 ...
43	BD391474.1.p1	BD391474.1.p1 GENE.BD391474.1~~BD391474.1.p1 ...
268	PP061819.1.p1	PP061819.1.p1 GENE.PP061819.1~~PP061819.1.p1 ...
251	PP061795.1.p1	PP061795.1.p1 GENE.PP061795.1~~PP061795.1.p1 ...
253	PP061808.1.p1	PP061808.1.p1 GENE.PP061808.1~~PP061808.1.p1 ...
263	PP061818.1.p1	PP061818.1.p1 GENE.PP061818.1~~PP061818.1.p1 ...
20	AJ300266.1.p3	AJ300266.1.p3 GENE.AJ300266.1~~AJ300266.1.p3 ...
277	PQ276083.1.p1	PQ276083.1.p1 GENE.PQ276083.1~~PQ276083.1.p1 ...
319	Z11741.1.p1	Z11741.1.p1 GENE.Z11741.1~~Z11741.1.p1 ORF ty...
22	AJ300266.1.p2	AJ300266.1.p2 GENE.AJ300266.1~~AJ300266.1.p2 ...

	protein_length	probabilidade_resistencia
45	849	0.999992
21	486	0.995776
259	337	0.988616
265	337	0.985265
250	337	0.985265
255	337	0.985265
273	337	0.982358
258	471	0.975159
272	478	0.974111
43	871	0.972653
268	337	0.972129
251	478	0.965847
253	478	0.961767
263	478	0.961767
20	174	0.924701
277	603	0.921982
319	510	0.913497
22	333	0.908519

VALIDAÇÃO BIOLÓGICA

P_LOOP : 9
KINASE2 : 2
RNBS_A : 1
RNBS_B : 5
RNBS_C : 2
GLPL : 8

RNBS_D : 10
MHD : 2

Predição para PHYSALIS

TOP CANDIDATOS com probabilidade de >= {probabilidade}:
pipelineW/physalis_transcriptoma.fasta, MODELO: SVM

	id		descricao	\
8851	J0132546.1.p1	J0132546.1.p1	GENE.J0132546.1~~J0132546.1.p1	...
12868	J0138683.1.p1	J0138683.1.p1	GENE.J0138683.1~~J0138683.1.p1	...
8853	J0132547.1.p1	J0132547.1.p1	GENE.J0132547.1~~J0132547.1.p1	...
8856	J0132548.1.p1	J0132548.1.p1	GENE.J0132548.1~~J0132548.1.p1	...
941	J0125565.1.p1	J0125565.1.p1	GENE.J0125565.1~~J0125565.1.p1	...
...	...	...	...	...
8637	J0131909.1.p1	J0131909.1.p1	GENE.J0131909.1~~J0131909.1.p1	...
12560	J0138486.1.p2	J0138486.1.p2	GENE.J0138486.1~~J0138486.1.p2	...
14263	J0139777.1.p1	J0139777.1.p1	GENE.J0139777.1~~J0139777.1.p1	...
8397	J0131706.1.p1	J0131706.1.p1	GENE.J0131706.1~~J0131706.1.p1	...
13186	J0138922.1.p2	J0138922.1.p2	GENE.J0138922.1~~J0138922.1.p2	...

	protein_length	probabilidade_resistencia
8851	776	1.000000
12868	1148	1.000000
8853	759	1.000000
8856	639	1.000000
941	646	0.999999
...	...	...
8637	335	0.853816
12560	396	0.852269
14263	236	0.851859
8397	430	0.850153
13186	230	0.850064

[291 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 111
KINASE2 : 106
RNBS_A : 26
RNBS_B : 60
RNBS_C : 48
GLPL : 62
RNBS_D : 75
MHD : 29

Arquivo salvo:

pipelineW/predicoes_svm.csv

```
[14]: print(df_validacao)
```

```
[{'P_LOOP': 34, 'KINASE2': 31, 'RNBS_A': 13, 'RNBS_B': 5, 'RNBS_C': 8, 'GLPL': 15, 'RNBS_D': 18, 'MHD': 18, 'planta': 'Arabidopsis thaliana', 'modelo': 'XGBoost'}, {'P_LOOP': 14, 'KINASE2': 33, 'RNBS_A': 13, 'RNBS_B': 25, 'RNBS_C': 18, 'GLPL': 16, 'RNBS_D': 16, 'MHD': 2, 'planta': 'Solanum melongena', 'modelo': 'XGBoost'}, {'P_LOOP': 10, 'KINASE2': 3, 'RNBS_A': 1, 'RNBS_B': 5, 'RNBS_C': 1, 'GLPL': 8, 'RNBS_D': 9, 'MHD': 2, 'planta': 'Solanum tuberosum', 'modelo': 'XGBoost'}, {'P_LOOP': 103, 'KINASE2': 132, 'RNBS_A': 27, 'RNBS_B': 46, 'RNBS_C': 36, 'GLPL': 60, 'RNBS_D': 71, 'MHD': 24, 'planta': 'Physalis peruviana', 'modelo': 'XGBoost'}, {'P_LOOP': 24, 'KINASE2': 19, 'RNBS_A': 11, 'RNBS_B': 1, 'RNBS_C': 6, 'GLPL': 14, 'RNBS_D': 9, 'MHD': 16, 'planta': 'Arabidopsis thaliana', 'modelo': 'RF'}, {'P_LOOP': 9, 'KINASE2': 15, 'RNBS_A': 4, 'RNBS_B': 7, 'RNBS_C': 0, 'GLPL': 11, 'RNBS_D': 13, 'MHD': 2, 'planta': 'Solanum melongena', 'modelo': 'RF'}, {'P_LOOP': 2, 'KINASE2': 2, 'RNBS_A': 1, 'RNBS_B': 0, 'RNBS_C': 0, 'GLPL': 1, 'RNBS_D': 2, 'MHD': 1, 'planta': 'Solanum tuberosum', 'modelo': 'RF'}, {'P_LOOP': 46, 'KINASE2': 50, 'RNBS_A': 6, 'RNBS_B': 15, 'RNBS_C': 8, 'GLPL': 47, 'RNBS_D': 35, 'MHD': 18, 'planta': 'Physalis peruviana', 'modelo': 'RF'}, {'P_LOOP': 33, 'KINASE2': 27, 'RNBS_A': 13, 'RNBS_B': 3, 'RNBS_C': 8, 'GLPL': 16, 'RNBS_D': 16, 'MHD': 17, 'planta': 'Arabidopsis thaliana', 'modelo': 'SVM'}, {'P_LOOP': 11, 'KINASE2': 23, 'RNBS_A': 11, 'RNBS_B': 20, 'RNBS_C': 16, 'GLPL': 13, 'RNBS_D': 22, 'MHD': 4, 'planta': 'Solanum melongena', 'modelo': 'SVM'}, {'P_LOOP': 9, 'KINASE2': 2, 'RNBS_A': 1, 'RNBS_B': 5, 'RNBS_C': 2, 'GLPL': 8, 'RNBS_D': 10, 'MHD': 2, 'planta': 'Solanum tuberosum', 'modelo': 'SVM'}, {'P_LOOP': 111, 'KINASE2': 106, 'RNBS_A': 26, 'RNBS_B': 60, 'RNBS_C': 48, 'GLPL': 62, 'RNBS_D': 75, 'MHD': 29, 'planta': 'Physalis peruviana', 'modelo': 'SVM'}]
```

```
[15]: df_copia = df_validacao.copy()
df_inicial = pd.DataFrame(df_validacao)

df_validacao = df_inicial.melt(
    id_vars=["planta", "modelo"],
    var_name="motivo",
    value_name="contagem"
)
print(df_validacao.head())

df_validacao.to_csv(f"{OUTPUT_DIR}/df_validacao.csv", index=False)
```

	planta	modelo	motivo	contagem
0	Arabidopsis thaliana	XGBoost	P_LOOP	34
1	Solanum melongena	XGBoost	P_LOOP	14
2	Solanum tuberosum	XGBoost	P_LOOP	10
3	Physalis peruviana	XGBoost	P_LOOP	103
4	Arabidopsis thaliana	RF	P_LOOP	24

```
[16]: #Scatterplot
plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df_validacao,
    x="motivo",
    y="contagem",
    hue="planta",
    style="modelo",
    s=200
)
# plt.title( "Validação biológica dos motivos conservados")
plt.ylabel("Número de ocorrências")
plt.xlabel("Motivo")
plt.legend(
    bbox_to_anchor=(1.05,1),
    loc="upper left"
)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/Validacao_biologica_dos_motivos_conservados.png")
plt.close()
# plt.show()

#Bubble Chart
plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df_validacao,
    x="motivo",
    y="modelo",
    hue="planta",
    size="contagem",
    sizes=(50,600)
)
# plt.title("Distribuição dos motivos por modelo e espécie")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/distribuicao_dos_motivos_por_modelo_especie.png")
plt.close()
# plt.show()

#Facetas por planta
g = sns.catplot(
    data=df_validacao,
    x="motivo",
    y="contagem",
    hue="modelo",
    col="planta",
    kind="bar",

```

```

        height=5,
        aspect=1.2
    )
    g.set_xticklabels(rotation=45)
    plt.savefig(f"{OUTPUT_DIR}/facetas_por_planta.png")
    plt.close()
    # plt.show()

#por planta

#Physalis peruviana
df_physalis = df_validacao[df_validacao["planta"] == "Physalis peruviana"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_physalis,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Physalis peruviana", fontsize=14,
#           fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_physalis.png", dpi=300)
plt.close()

#Solanum melongena
df_melongena = df_validacao[df_validacao["planta"] == "Solanum melongena"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_melongena,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos

```

```

    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Solanum melongena", fontsize=14,
# ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_melongena.png", dpi=300)
plt.close()

#Solanum tuberosum
df_tuberosum = df_validacao[df_validacao["planta"] == "Solanum tuberosum"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_tuberosum,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Solanum tuberosum", fontsize=14,
# ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_tuberosum.png", dpi=300)
plt.close()

#Heatmap
df_validacao["grupo"] = (
    df_validacao["planta"]
    + "_"
    + df_validacao["modelo"]

```

```

)

pivot = df_validacao.pivot_table(
    index="motivo",
    columns="grupo",
    values="contagem",
    fill_value=0
)

plt.figure(figsize=(14, 8))
sns.heatmap(
    pivot,
    annot=True,
    fmt="g",
    cmap="YlGnBu",
    linewidths=0.5,
    cbar_kws={'label': 'Contagem'} # Nomeia a barra de cores lateral
)

# plt.title("Distribuição dos Motivos Conservados por Planta e Modelo",
#           ↪fontsize=14, fontweight="bold", pad=15)
plt.ylabel("Motivo", fontsize=12)
plt.xlabel("Grupo (Planta & Modelo)", fontsize=12)
plt.xticks(rotation=45, ha="right", fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/heatmap_distribuicao_dos_motivos_conservados.
           ↪png", dpi=300)
plt.close()
# plt.show()

print("===== Fim do Pipeline =====")

```

```
===== Fim do Pipeline =====
```

```
[ ]:
```